



ELEC 5690 - Advanced Topics in Artificial Intelligence for Medical Image Analysis

Lecture 01b: Deep Learning Fundamentals

Sep 1, 2026



- Review: Machine learning vs. deep learning framework
- Deep learning basics through a simple example
 - Defining a neural network architecture
 - Defining a loss function
 - Optimizing the loss function
- Model implementation using deep learning frameworks
- Design choices

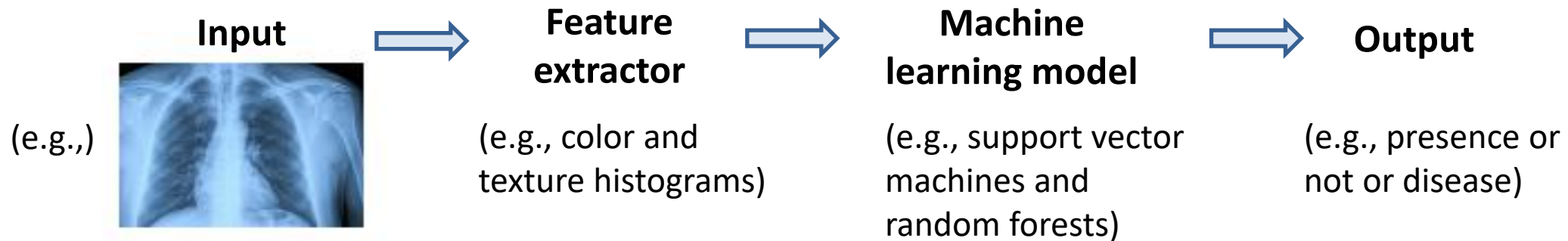
1. Machine learning vs. deep learning framework



- Machine Learning Framework

Data-driven learning of a mapping from input to output

Traditional machine learning approaches

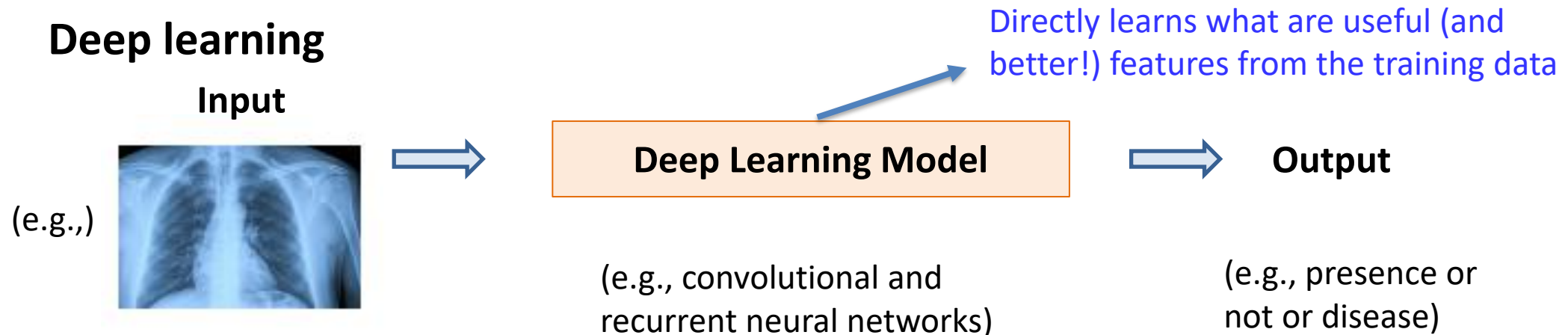
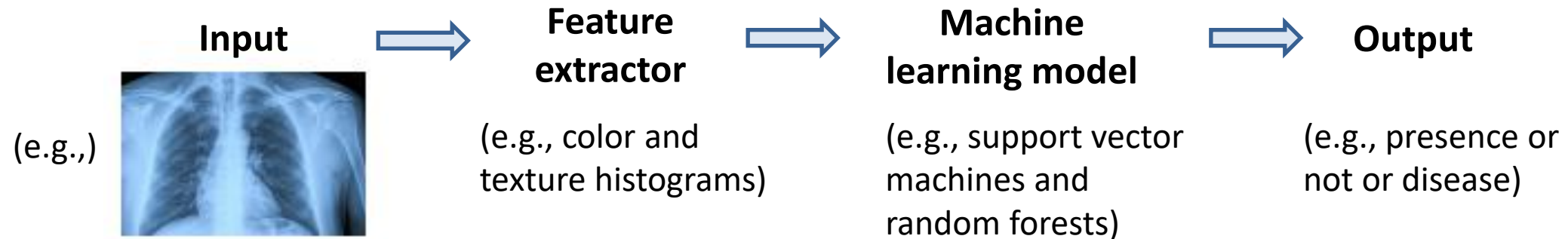


1. Machine learning vs. deep learning framework



- Deep learning (a type of machine learning)

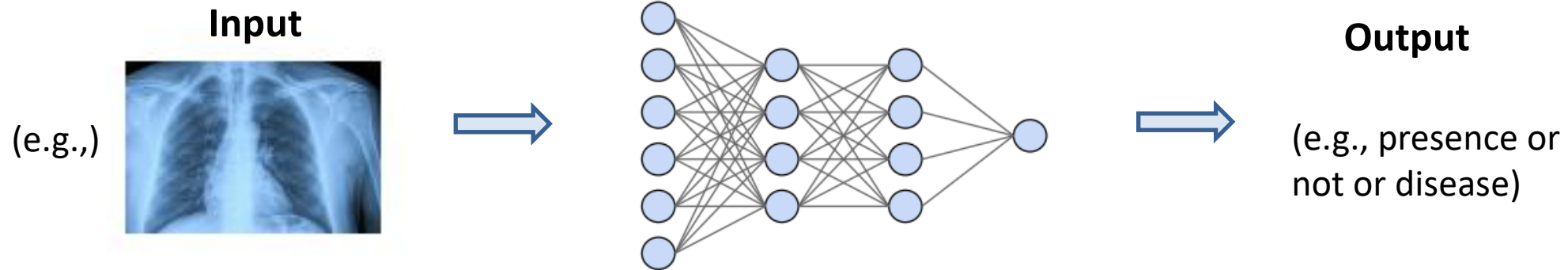
Traditional machine learning approaches



1. Machine learning vs. deep learning framework



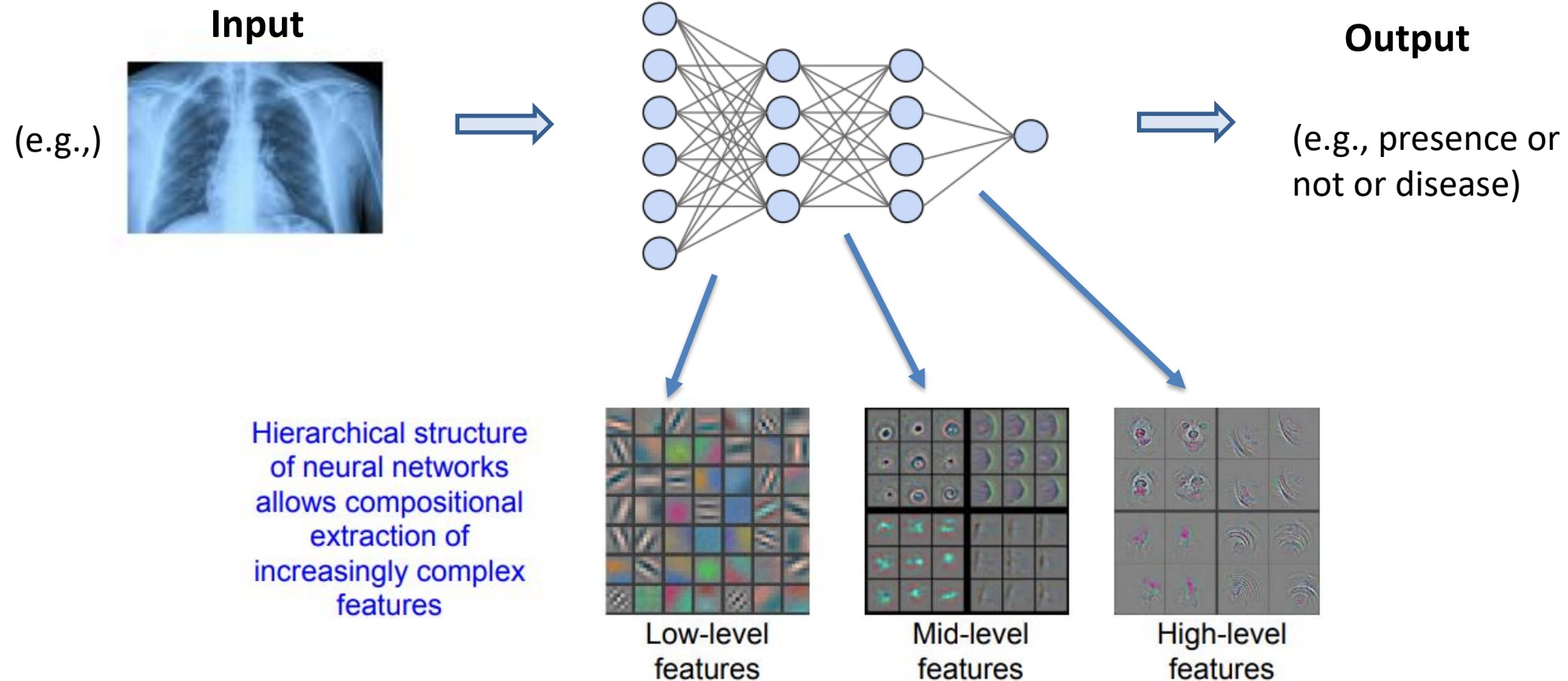
- How do deep learning models perform feature extraction?



1. Machine learning vs. deep learning framework



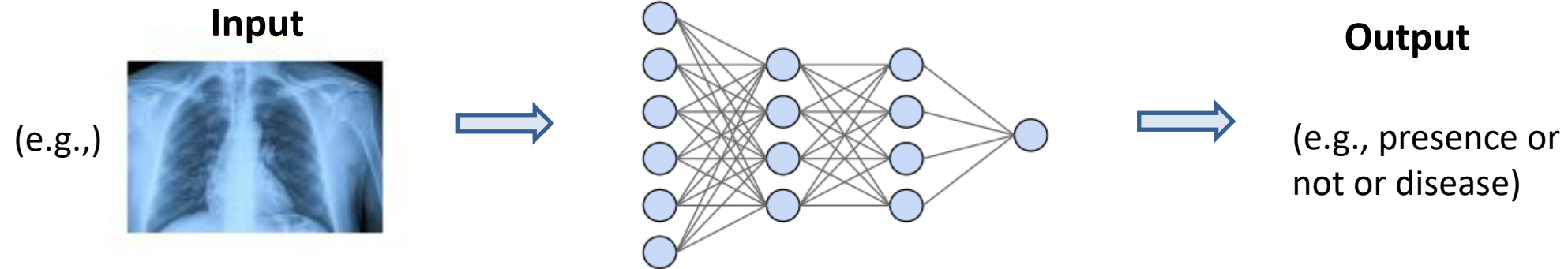
- How do deep learning models perform feature extraction?



1. Machine learning vs. deep learning framework



- How do deep learning models perform feature extraction?



1. Preparing data for deep learning
2. Neural network models
3. Training neural networks
4. Evaluating models

2. Deep learning basics through a simple example

Let us consider the task of regression:

Predicting a single real-valued output from input data

Model input: input x

Model output: prediction (single number) $\hat{y}$

BMD is important for diagnosing osteoporosis, a condition characterized by decreased levels of bone density.

Example: Estimating bone mineral density (BMD) from X-ray images

$x \in R^{H \times W}$ an X-ray image with size of $H \times W$

$y = BMD \text{ value}$

Example: Predicting left-ventricular ejection fraction (LVEF) from echocardiogram videos

$x \in R^{H \times W \times T}$ an echocardiogram video

$y = LVEF \text{ value}$

LVEF is a percentage measurement of how much blood the left ventricle pumps out with each contraction, serving as an essential indicator to diagnose and track heart failure.

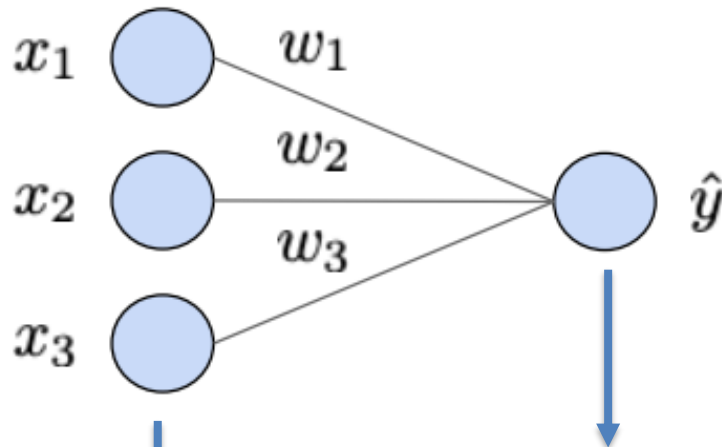
2. Deep learning basics through a simple example

- Defining a neural network architecture

Our first architecture: a **single-layer, fully connected** neural network

For simplicity, use a 3-dimensional input ($N = 3$)

All inputs of a layer are connected to all outputs of a layer



Layer inputs

Layer output(s)

Output: $\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$
 $= w^T x + b$

bias term (allows constant shift)

Neural network parameters:

$$W = \{[w_1, w_2, w_3], b\}$$

Layer weights

Layer bias⁹

Often refer to all parameters together as just “weights”. Bias is implicitly assumed.

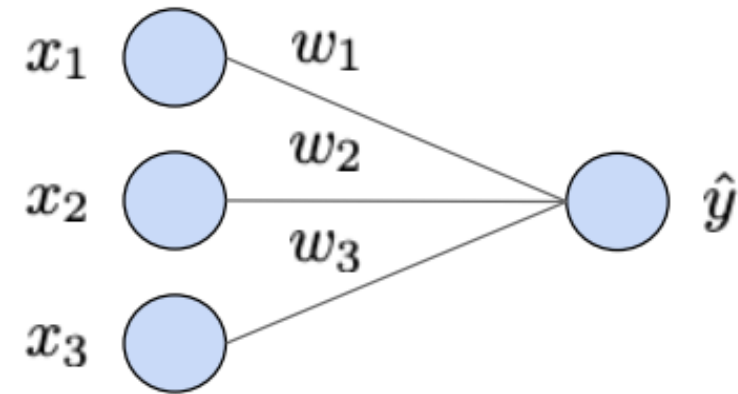
2. Deep learning basics through a simple example

- Defining a loss function

Output: $\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$
 $= w^T x + b$

Neural network parameters:

$$W = \{[w_1, w_2, w_3], b\}$$



Loss functions are quantitative measures of how satisfactory the model predictions are (i.e., how “good” the model parameters are).

Mean square error (MSE) loss, which is standard for regression

MSE loss for a single example x^i , when the prediction is $\hat{y}^i$ and the ground-truth is y^i

$$L^i(W) = (\hat{y}^i - y^i)^2 \quad \leftarrow \text{the loss is small when the prediction is close to the ground truth}$$

MSE loss over a set of examples $i = \{1, \dots, M\}$: $L = \frac{1}{M} \sum_i L^i(W)$

2. Deep learning basics through a simple example

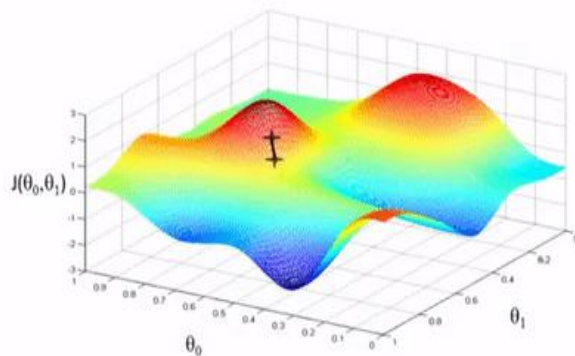


- Optimizing the loss function: gradient descent

Goal: find the best values for the model parameters that minimize the loss function.

The approach we will take **gradient descent**

Main idea: iteratively update the model parameters, to take steps in the local direction of steepest (negative) slope, i.e., the negative gradient

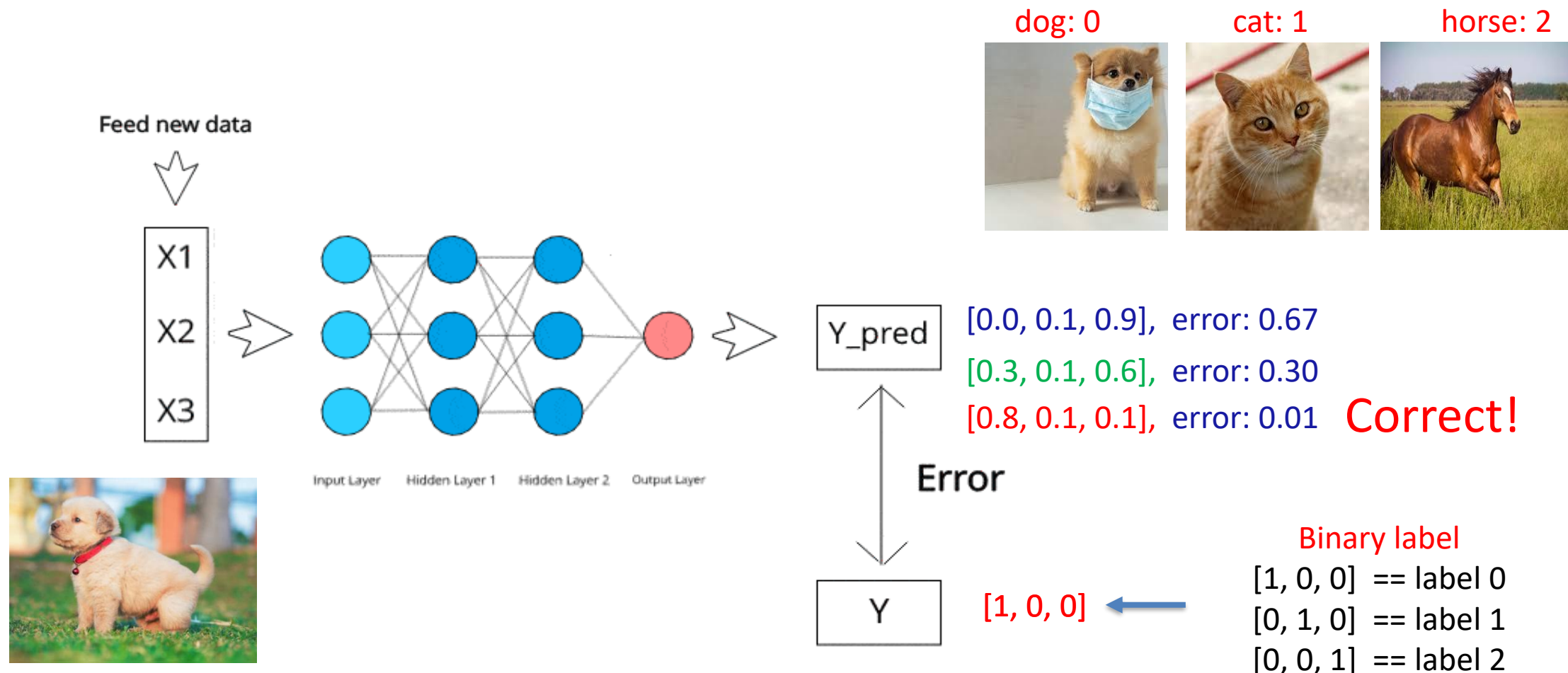


We will be able to use gradient descent to iteratively optimize the complex loss function landscapes corresponding to deep neural networks.

Andrew Ng

2. Deep learning basics through a simple example

- Convolutional neural network (CNN) – how to train?



2. Deep learning basics through a simple example

- Review from calculus: derivatives and gradients

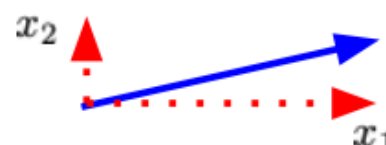
The **derivative** of a function is a measure of local slope.

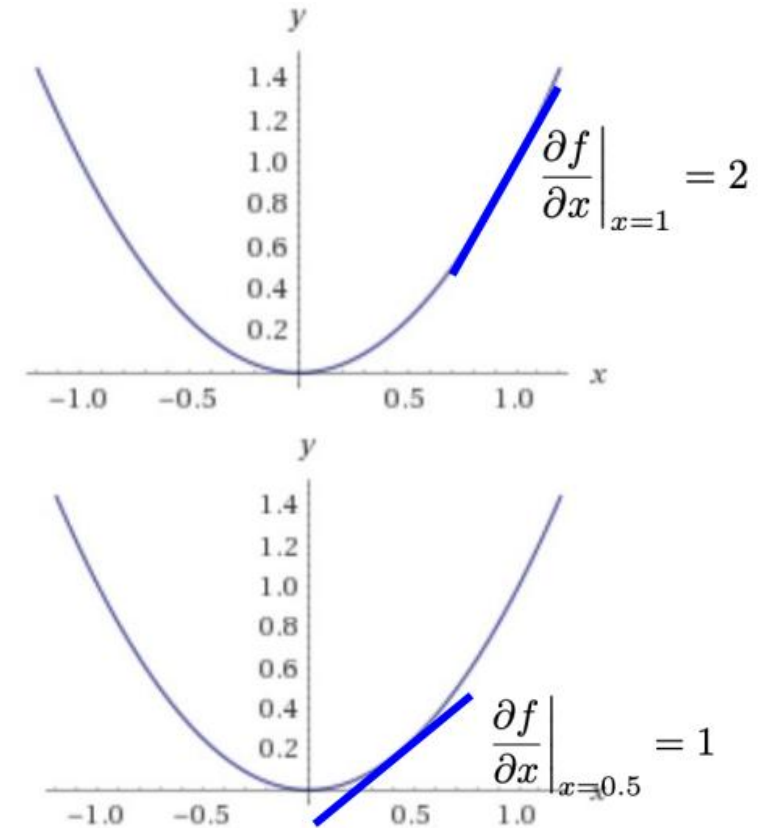
$$\text{Ex: } f(x) = x^2, \quad \frac{\partial f}{\partial x} = 2x$$

The **derivative** of a of multiple variables is the vector of partial derivatives of the function with respect to each variable.

$$\text{Ex: } f(x) = 3x_1^2 + x_2^2, \quad \nabla f_x = [6x_1, 2x_2]$$

The gradient evaluated at a particular point is the direction of steepest ascent of the function.

$$\nabla f_x \Big|_{x_1=1, x_2=1} = [6, 2]$$




The negative of the gradient is the direction of steepest descent -> direction we want to move in the loss function landscape!

2. Deep learning basics through a simple example

- Gradient descent algorithm

Let the gradient of the loss function with respect to the model parameters w be:

$$\nabla L_w = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_k} \right]$$

Then we can minimize the loss function by iteratively updating the model parameters (“taking steps”) **in the direction of the negative gradient**, until convergence:

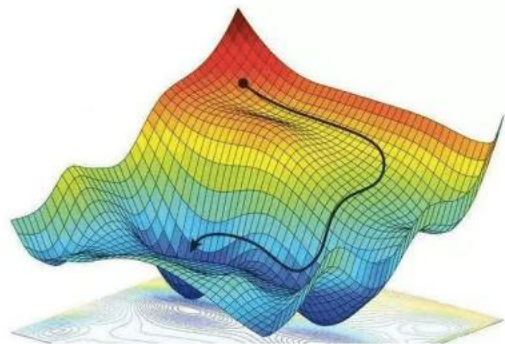
$$W := W - \alpha \nabla L_W$$

“Step size” hyperparameter (design choice)
indicating **how big of a step in the negative gradient direction** we want to take at each update.

Too big -> may overshoot minima.
Too small -> optimization takes too long.

For ease of notation, rewrite parameter b as w_0 corresponding to $x_0 = 1$:

$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2 + w_3 x_3$$
$$W = \{[w_0, w_1, w_2, w_3]\}$$



2. Deep learning basics through a simple example

- Gradient descent algorithm: in code

```
# initialize vector of weight parameters to random values
weights = random_init(weights_dimension)

while True:
    # evaluate the gradient of the loss function with respect to the weights
    weights_grad = evaluate_gradient(loss_fcn, data, weights)
    # update the weights in the direction of the negative gradient
    weights = weights - step_size * weights_grad
```

2. Deep learning basics through a simple example

- Stochastic gradient descent (SGD)

Evaluating gradient involves iterating over all data examples, which can be slow!

In practice, usually use stochastic gradient descent: estimate gradient over a sample of data examples (usually as many as can fit on GPU at one time, e.g. 32, 64, 128)

```
# initialize vector of weight parameters to random values
weights = random_init(weights_dimension)

while True:

    # sample a batch of data examples
    data_batch = sample_data(data, 128)

    # evaluate the gradient of the loss function with respect to the weights
    weights_grad = evaluate_gradient(loss_fcn, data_batch, weights)
    # update the weights in the direction of the negative gradient
    weights = weights - step_size * weights_grad
```

Differences:

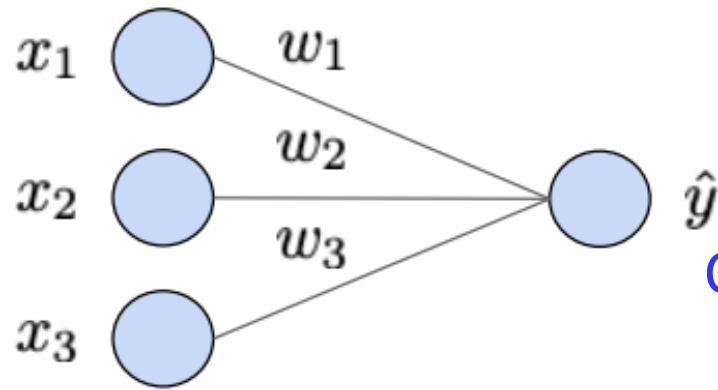
GD - You have to run through **ALL the samples in your training** set to do a single update for a parameter in a particular iteration.

SGD - You use **ONLY ONE or SUBSET of training sample from your training set** to do the update for a parameter in a particular iteration.

2. Deep learning basics through a simple example

- Optimizing the loss function:

Our example:



Output: $\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$
 $= w^T x + b$

Neural network parameters:

$$W = \{[w_1, w_2, w_3], b\}$$

Loss function:

Per-example: $L^i(W) = (\hat{y}^i - y^i)^2$

Over M examples: $L = \frac{1}{M} \sum_i L^i(W)$

Gradient of loss w.r.t. weights:

Partial derivative of loss w.r.t. kth weight:

Chain rule $\rightarrow$ $\frac{\partial L^i}{\partial w_k} = \frac{\partial L^i}{\partial \hat{y}^i} \frac{\partial \hat{y}^i}{\partial w_k} = 2(\hat{y}^i - y^i)x_k^i$

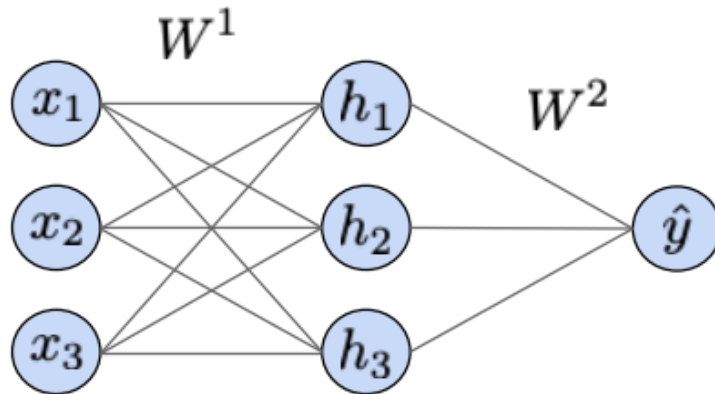
Over M examples $\rightarrow$ $\frac{\partial L}{\partial w_k} = \frac{1}{M} \sum_i \frac{\partial L^i}{\partial w_k} = \frac{1}{M} \sum_i 2(\hat{y}^i - y^i)x_k^i$

Full gradient expression:

$$\nabla L_W = \left[\frac{\partial L}{\partial w_0}, \dots, \frac{\partial L}{\partial w_3} \right] = \boxed{\frac{1}{M} \sum_i 2(\hat{y}^i - y^i)x^i}$$

2. Deep learning basics through a simple example

- Now: a two-layer fully-connected neural network



$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$

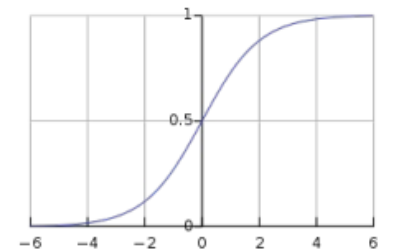
$$W^2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \end{bmatrix} \quad b^2 = \begin{bmatrix} b_1^2 \end{bmatrix}$$

Output: $h = \sigma(W^1 x + b^1)$
 $\hat{y} = W^2 h + b^2$

$\sigma(a) = \frac{1}{1 + e^{-a}}$

Full function expression:

$$\hat{y} = W^2(\sigma(W^1 x + b^1)) + b^2$$



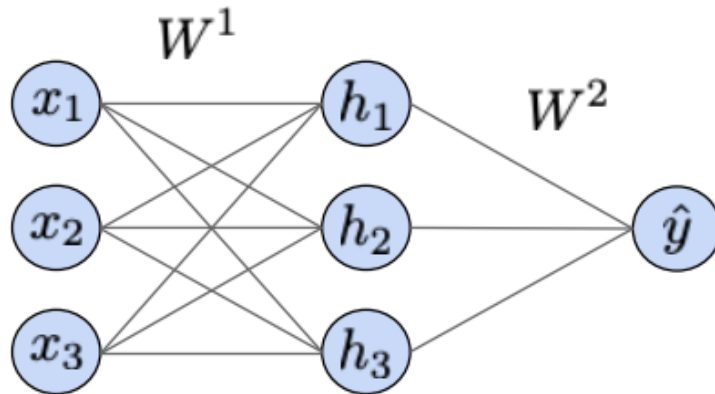
Sigmoid "activation function"

Activation functions

Introduce non-linearity into the model -- allowing it to represent highly complex functions.

2. Deep learning basics through a simple example

- Now: a two-layer fully-connected neural network

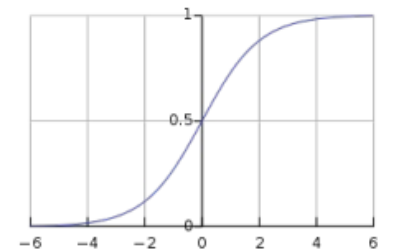


Output: $h = \sigma(W^1 x + b^1)$
 $\hat{y} = W^2 h + b^2$

$\sigma(a) = \frac{1}{1 + e^{-a}}$

Full function expression:

$$\hat{y} = W^2(\sigma(W^1 x + b^1)) + b^2$$



Sigmoid "activation function"

Activation functions

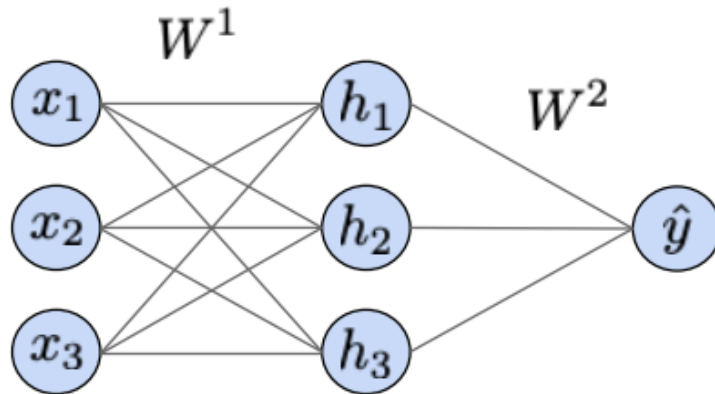
Introduce non-linearity into the model --
allowing it to represent highly complex functions.

$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$

$$W^2 = [w_{11}^2 \quad w_{12}^2 \quad w_{13}^2] \quad b^2 = [b_1^2]$$

2. Deep learning basics through a simple example

- Now: a two-layer fully-connected neural network

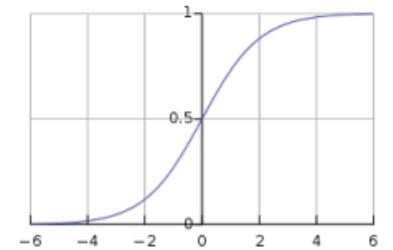


Output: $h = \sigma(W^1 x + b^1)$
 $\hat{y} = W^2 h + b^2$

$\sigma(a) = \frac{1}{1 + e^{-a}}$

Full function expression:

$$\hat{y} = W^2(\sigma(W^1 x + b^1)) + b^2$$



Sigmoid "activation function"

Activation functions

Introduce non-linearity into the model --
allowing it to represent highly complex functions.

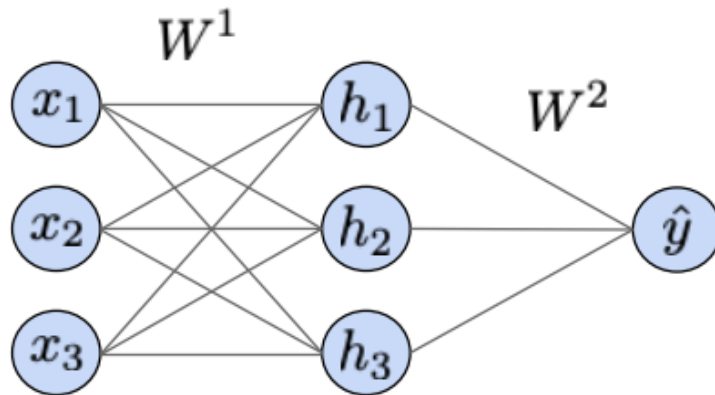
A fully-connected neural network (also known as multi-layer perceptron) is a stack of [affine transformation + activation function] layers. There may not be an activation function in the last layer

$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$

$$W^2 = [w_{11}^2 \quad w_{12}^2 \quad w_{13}^2] \quad b^2 = [b_1^2]$$

2. Deep learning basics through a simple example

- Now: a two-layer fully-connected neural network



$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$

$$W^2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \end{bmatrix} \quad b^2 = \begin{bmatrix} b_1^2 \end{bmatrix}$$

Output: $\hat{y} = W^2(\sigma(W^1x + b^1)) + b^2$

Neural network parameters:

$$W = \{W^1, b^1, W^2, b^2\}$$

Loss function (regression loss, same as before):

Per-example: $L^i(W) = (\hat{y}^i - y^i)^2$

Over M examples: $L = \frac{1}{M} \sum_i L^i(W)$

Gradient of loss w.r.t. weights:

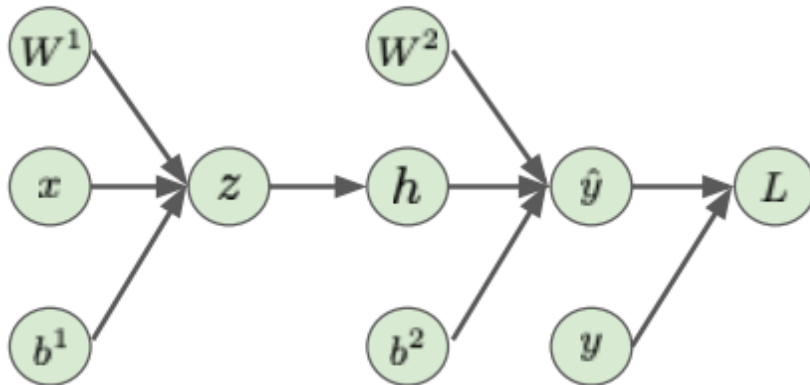
Function more complex -> now much harder to derive the expressions! Instead... computational graphs and backpropagation.

2. Deep learning basics through a simple example

- Computing gradients with backpropagation

Network output: $\hat{y} = W^2(\sigma(W^1x + b^1)) + b^2$

Think of computing loss function as staged computation of intermediate variables:



“Forward pass”:

$$z = W^1x + b^1$$
$$h = \sigma(z)$$
$$\hat{y} = W^2h + b^2$$
$$L = (\hat{y} - y)^2$$

Now, can use a repeated application of the chain rule, going backwards through the computational graph, to obtain the gradient of the loss with respect to each node of the computation graph.

“Backward pass”:

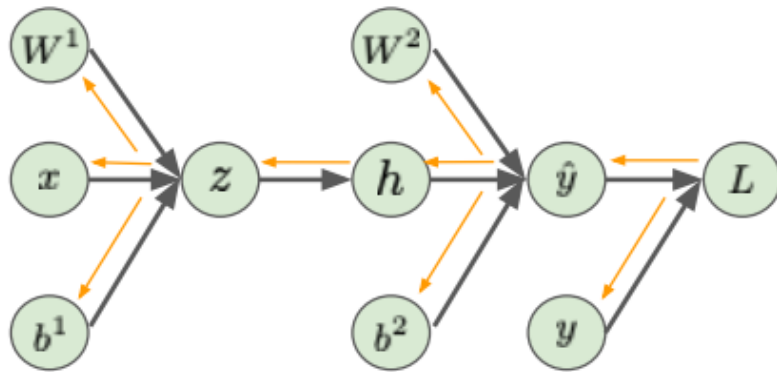
$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) \quad (\text{not all gradients shown})$$
$$\frac{\partial L}{\partial W^2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W^2}$$
$$\frac{\partial L}{\partial H} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial H}$$
$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial H} \frac{\partial H}{\partial Z}$$
$$\frac{\partial L}{\partial W^1} = \frac{\partial L}{\partial Z} \frac{\partial Z}{\partial W^1}$$

2. Deep learning basics through a simple example

- Computing gradients with backpropagation

Network output: $\hat{y} = W^2(\sigma(W^1x + b^1)) + b^2$

Think of computing loss function as staged computation of intermediate variables:



“Forward pass”:

$$z = W^1x + b^1$$
$$h = \sigma(z)$$
$$\hat{y} = W^2h + b^2$$
$$L = (\hat{y} - y)^2$$

Now, can use a repeated application of the chain rule, going backwards through the computational graph, to obtain the gradient of the loss with respect to each node of the computation graph.

Plug in from earlier computations via chain rule

“Backward pass”:

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) \quad (\text{not all gradients shown})$$
$$\frac{\partial L}{\partial W^2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W^2}$$
$$\frac{\partial L}{\partial H} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial H}$$
$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial H} \frac{\partial H}{\partial Z}$$
$$\frac{\partial L}{\partial W^1} = \frac{\partial L}{\partial Z} \frac{\partial Z}{\partial W^1}$$

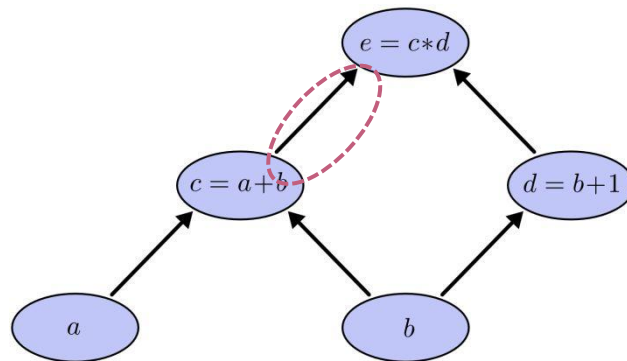
Local gradients to derive

2. Deep learning basics through a simple example

- Computing gradients with backpropagation

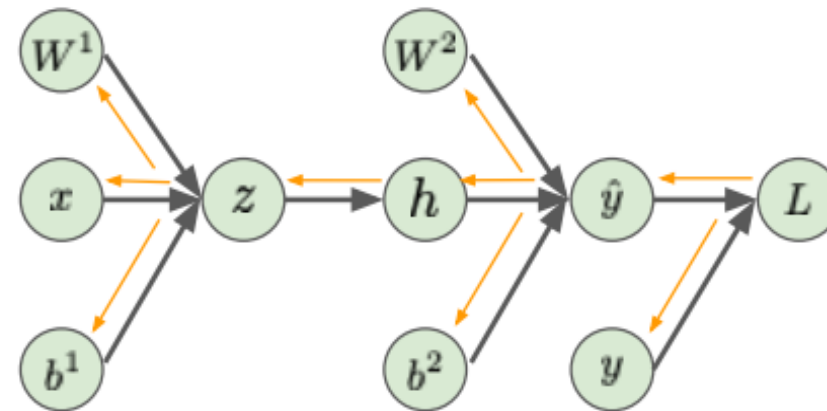
Key idea: Don't mathematically derive entire math expression for e.g. dL / dW^1 . By writing it as nested applications of the chain rule, only have to derive simple “local” gradients representing relationships between connected nodes of the graph (e.g. dH / dW^1).

Can use more or less intermediate variables to control how difficult local gradients are to derive!



$$\frac{\partial e}{\partial a} = \frac{\partial e}{\partial c} \cdot \frac{\partial c}{\partial a} \quad \frac{\partial e}{\partial b} = \frac{\partial e}{\partial c} \cdot \frac{\partial c}{\partial b} + \frac{\partial e}{\partial d} \cdot \frac{\partial d}{\partial b}$$

a demand salary for c , c demand salary for e



backpropagation

e pay for c , c pay for a – much more efficient

2. Deep learning basics through a simple example

Training our two-layer neural network in code, using backpropagation

```
# initialize model parameters to be learned
W1 = np.random.rand(input_dim, hid_dim)
W2 = np.random.rand(hid_dim, output_dim)
b1 = np.random.rand(1, hid_dim)
b2 = np.random.rand(1, output_dim)

# perform gradient descent
step_size = 1e-2

while(keep_training)

    # forward pass, computing loss
    Z_curr = X.dot(W1) + b1
    H_curr = sigmoid_array(Z_curr)
    Y_curr = H_curr.dot(W2) + b2
    loss = np.sum(np.square(Y_curr - Y)) / num_examples

    # backward pass, computing gradients of loss with respect to each
    # variable in the computation graph
    d_Y_curr = 2*(Y_curr - Y) / num_examples
    d_H_curr = d_Y_curr.dot(W2.T)
    d_W2 = H_curr.T.dot(d_Y_curr)
    d_b2 = d_Y_curr
    d_Z_curr = d_H_curr * sigmoid_array(Z_curr)*(1-sigmoid_array(Z_curr))
    d_X = d_Z_curr.dot(W1.T)
    d_W1 = d_X.T.dot(d_Z_curr)
    d_b1 = d_Y_curr

    # perform gradient update
    W1 = W1 - step_size * d_W1
    b1 = b1 - step_size * d_b1
    W2 = W2 - step_size * d_W2
    b2 = b2 - step_size * d_b2
```

2. Deep learning basics through a simple example

Training our two-layer neural network in code, using backpropagation

```
# initialize model parameters to be learned
W1 = np.random.rand(input_dim, hid_dim)
W2 = np.random.rand(hid_dim, output_dim)
b1 = np.random.rand(1, hid_dim)
b2 = np.random.rand(1, output_dim)

# perform gradient descent
step_size = 1e-2

while(keep_training)

    # forward pass, computing loss
    Z_curr = X.dot(W1) + b1
    H_curr = sigmoid_array(Z_curr)
    Y_curr = H_curr.dot(W2) + b2
    loss = np.sum(np.square(Y_curr - Y)) / num_examples

    # backward pass, computing gradients of loss with respect to each
    # variable in the computation graph
    d_Y_curr = 2*(Y_curr - Y) / num_examples
    d_H_curr = d_Y_curr.dot(W2.T)
    d_W2 = H_curr.T.dot(d_Y_curr)
    d_b2 = d_Y_curr
    d_Z_curr = d_H_curr * sigmoid_array(Z_curr)*(1-sigmoid_array(Z_curr))
    d_X = d_Z_curr.dot(W1.T)
    d_W1 = d_X.T.dot(d_Z_curr)
    d_b1 = d_Y_curr

    # perform gradient update
    W1 = W1 - step_size * d_W1
    b1 = b1 - step_size * d_b1
    W2 = W2 - step_size * d_W2
    b2 = b2 - step_size * d_b2
```

} Initialize model parameters

2. Deep learning basics through a simple example

Training our two-layer neural network in code, using backpropagation

```
# initialize model parameters to be learned
W1 = np.random.rand(input_dim, hid_dim)
W2 = np.random.rand(hid_dim, output_dim)
b1 = np.random.rand(1, hid_dim)
b2 = np.random.rand(1, output_dim)

# perform gradient descent
step_size = 1e-2

while(keep_training)

    # forward pass, computing loss
    Z_curr = X.dot(W1) + b1
    H_curr = sigmoid_array(Z_curr)
    Y_curr = H_curr.dot(W2) + b2
    loss = np.sum(np.square(Y_curr - Y)) / num_examples

    # backward pass, computing gradients of loss with respect to each
    # variable in the computation graph
    d_Y_curr = 2*(Y_curr - Y) / num_examples
    d_H_curr = d_Y_curr.dot(W2.T)
    d_W2 = H_curr.T.dot(d_Y_curr)
    d_b2 = d_Y_curr
    d_Z_curr = d_H_curr * sigmoid_array(Z_curr)*(1-sigmoid_array(Z_curr))
    d_X = d_Z_curr.dot(W1.T)
    d_W1 = d_X.T.dot(d_Z_curr)
    d_b1 = d_Y_curr

    # perform gradient update
    W1 = W1 - step_size * d_W1
    b1 = b1 - step_size * d_b1
    W2 = W2 - step_size * d_W2
    b2 = b2 - step_size * d_b2
```

} Forward pass

2. Deep learning basics through a simple example

Training our two-layer neural network in code, using backpropagation

```
# initialize model parameters to be learned
W1 = np.random.rand(input_dim, hid_dim)
W2 = np.random.rand(hid_dim, output_dim)
b1 = np.random.rand(1, hid_dim)
b2 = np.random.rand(1, output_dim)

# perform gradient descent
step_size = 1e-2

while(keep_training)

    # forward pass, computing loss
    Z_curr = X.dot(W1) + b1
    H_curr = sigmoid_array(Z_curr)
    Y_curr = H_curr.dot(W2) + b2
    loss = np.sum(np.square(Y_curr - Y)) / num_examples

    # backward pass, computing gradients of loss with respect to each
    # variable in the computation graph
    d_Y_curr = 2*(Y_curr - Y) / num_examples
    d_H_curr = d_Y_curr.dot(W2.T)
    d_W2 = H_curr.T.dot(d_Y_curr)
    d_b2 = d_Y_curr
    d_Z_curr = d_H_curr * sigmoid_array(Z_curr)*(1-sigmoid_array(Z_curr))
    d_X = d_Z_curr.dot(W1.T)
    d_W1 = d_X.T.dot(d_Z_curr)
    d_b1 = d_Y_curr

    # perform gradient update
    W1 = W1 - step_size * d_W1
    b1 = b1 - step_size * d_b1
    W2 = W2 - step_size * d_W2
    b2 = b2 - step_size * d_b2
```

} Backward pass

2. Deep learning basics through a simple example

Training our two-layer neural network in code, using backpropagation

```
# initialize model parameters to be learned
W1 = np.random.rand(input_dim, hid_dim)
W2 = np.random.rand(hid_dim, output_dim)
b1 = np.random.rand(1, hid_dim)
b2 = np.random.rand(1, output_dim)

# perform gradient descent
step_size = 1e-2

while(keep_training)

    # forward pass, computing loss
    Z_curr = X.dot(W1) + b1
    H_curr = sigmoid_array(Z_curr)
    Y_curr = H_curr.dot(W2) + b2
    loss = np.sum(np.square(Y_curr - Y)) / num_examples

    # backward pass, computing gradients of loss with respect to each
    # variable in the computation graph
    d_Y_curr = 2*(Y_curr - Y) / num_examples
    d_H_curr = d_Y_curr.dot(W2.T)
    d_W2 = H_curr.T.dot(d_Y_curr)
    d_b2 = d_Y_curr
    d_Z_curr = d_H_curr * sigmoid_array(Z_curr)*(1-sigmoid_array(Z_curr))
    d_X = d_Z_curr.dot(W1.T)
    d_W1 = d_X.T.dot(d_Z_curr)
    d_b1 = d_Y_curr

    # perform gradient update
    W1 = W1 - step_size * d_W1
    b1 = b1 - step_size * d_b1
    W2 = W2 - step_size * d_W2
    b2 = b2 - step_size * d_b2
```

} Gradient update

- Who is the teaching assistant of this course?



(a)



(b)



(c)



- How to contact our TA?
 - a. Go to Rm 3116 during office hour
 - b. Go to Rm 3117 during office hour
 - c. Go to Tower B
 - d. Send an email to TA



- How to contact our TA?
 - a. Go to Rm 3116 during office hour
 - b. Go to Rm 3117 during office hour
 - c. Go to Tower B
 - d. Send an email to TA



- We have showed two examples of regression in medical image analysis. Could you please identify what those examples are?
 - a. Age estimation from X-rays
 - b. Skin lesion estimation from ultrasound
 - c. Bone density estimation from X-rays
 - d. left-ventricular ejection fraction prediction from echocardiogram videos
 - e. Blood pressure prediction from ultrasound



- We have showed two examples of regression in medical image analysis. Could you please identify what those examples are?
 - a. Age estimation from X-rays
 - b. Skin lesion estimation from ultrasound
 - c. Bone density estimation from X-rays
 - d. left-ventricular ejection fraction prediction from echocardiogram videos
 - e. Blood pressure prediction from ultrasound

3. Model implementation using frameworks



- Makes our lives easier by providing implementations and higher-level abstractions of many components for deep learning, and running them on GPUs:
 - Dataset batching, model definition, gradient computation, optimization, etc.
- Automatic differentiation: if we define nodes in a computational graph, will automatically implement backpropagation for us
 - Supports many common operations with local gradients already implemented
 - Can still define custom operations
- A number of popular options, e.g. Tensorflow and PyTorch.

3. Model implementation using frameworks



Training our two-layer neural network in code, in Tensorflow 2.0

```
# Our (X,Y) training set converted to TF tensors
X_tf = tf.convert_to_tensor(X, np.float32)
Y_tf = tf.convert_to_tensor(Y, np.float32)
# Create a TF dataset with specified minibatch size
batch_size = 50
dataset = tf.data.Dataset.from_tensor_slices((X_tf, Y_tf))
dataset = dataset.batch(batch_size)

# initialize model parameters to be learned
W1 = tf.Variable(tf.random.uniform((input_dim, hid_dim)))
W2 = tf.Variable(tf.random.uniform((hid_dim, output_dim)))
b1 = tf.Variable(tf.random.uniform((1, hid_dim)))
b2 = tf.Variable(tf.random.uniform((1, output_dim)))

# perform gradient descent
epochs = 5000
optimizer = tf.optimizers.SGD(learning_rate=1e-2)
losses = []

for epoch in range(epochs):
    for batch in dataset:
        X_batch, Y_batch = batch
        with tf.GradientTape() as tape:

            # forward pass
            Z_batch = tf.add(tf.matmul(X_batch, W1), b1)
            H_batch = tf.math.sigmoid(Z_batch)
            Out_batch = tf.add(tf.matmul(H_batch, W2), b2)
            loss = tf.losses.MSE(Y_batch, Out_batch)

            # backward pass and gradient update
            gradients = tape.gradient(loss, [W1, W2, b1, b2])
            optimizer.apply_gradients(zip(gradients, [W1, W2, b1, b2]))
```

Covert data to TF tensors, create a TF dataset

Initialize parameters to be learned as tf.Variable -> allows them to receive gradient updates during optimization

Initialize a TF optimizer

All operations defined under the gradient tape will be used to construct a computational graph

The computational graph for our two-layer neural network

Evaluate gradients using automatic differentiation and perform gradient update

3. Model implementation using frameworks



Also high level libraries built on top of Tensorflow, that provide even easier-to-use APIs:

In Tensorflow 2.0:

```
for epoch in range(epochs):
    for batch in dataset:
        X_batch, Y_batch = batch
        with tf.GradientTape() as tape:

            # forward pass
            Z_batch = tf.add(tf.matmul(X_batch, W1), b1)
            H_batch = tf.math.sigmoid(Z_batch)
            Out_batch = tf.add(tf.matmul(H_batch, W2), b2)
            loss = tf.losses.MSE(Y_batch, Out_batch)

            # backward pass and gradient update
            gradients = tape.gradient(loss, [W1, W2, b1, b2])
            optimizer.apply_gradients(zip(gradients, [W1, W2, b1, b2]))
```

In Keras:

```
keras_model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=1, use_bias=True)
])
keras_model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
                    loss='mse')
keras_model.fit(dataset, epochs=1000)
```

3. Model implementation using frameworks



Also high level libraries built on top of Tensorflow, that provide even easier-to-use APIs:

In Tensorflow 2.0:

```
for epoch in range(epochs):
    for batch in dataset:
        X_batch, Y_batch = batch
        with tf.GradientTape() as tape:

            # forward pass
            Z_batch = tf.add(tf.matmul(X_batch, W1), b1)
            H_batch = tf.math.sigmoid(Z_batch)
            Out_batch = tf.add(tf.matmul(H_batch, W2), b2)
            loss = tf.losses.MSE(Y_batch, Out_batch)

            # backward pass and gradient update
            gradients = tape.gradient(loss, [W1, W2, b1, b2])
            optimizer.apply_gradients(zip(gradients, [W1, W2, b1, b2]))
```

In Keras:

```
keras_model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=1, use_bias=True)
])
keras_model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
                    loss='mse')
keras_model.fit(dataset, epochs=1000)
```

Stack of layers

Fully-connected layer

Activation function and bias configurations included!

Specify hyperparameters for the training procedure

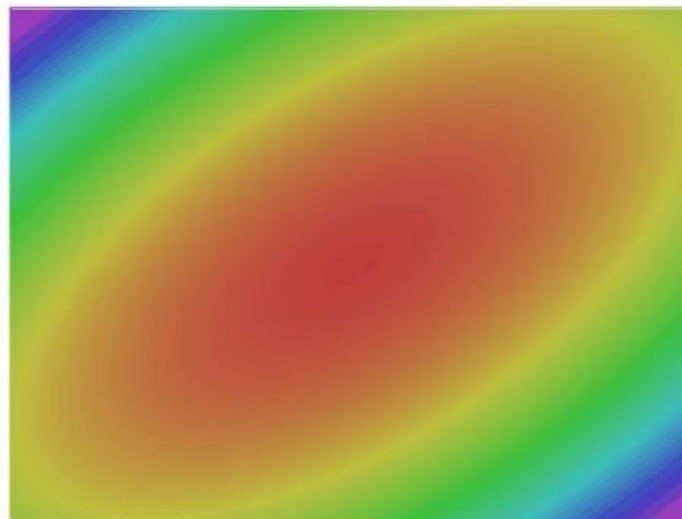
4. Design choices



Training hyperparameters: control knobs for the art of training neural networks

Optimization methods: SGD, SGD with momentum, RMSProp, Adam

- **Adam** is a good default choice in many cases; it often works ok even with constant learning rate
- **SGD+Momentum** can outperform Adam but may require more tuning of LR and schedule

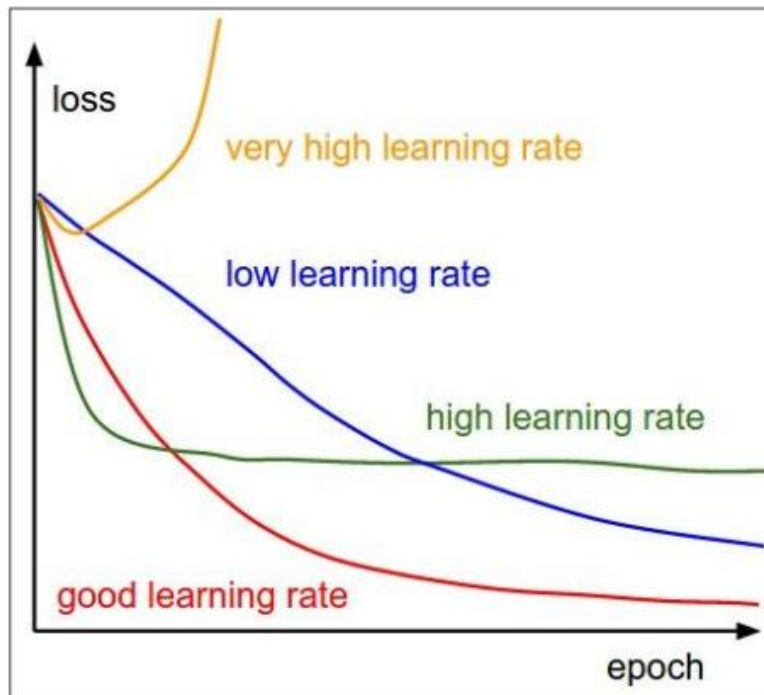


- SGD
- SGD+Momentum
- RMSProp
- Adam

4. Design choices



SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.



Slide credit: CS231n

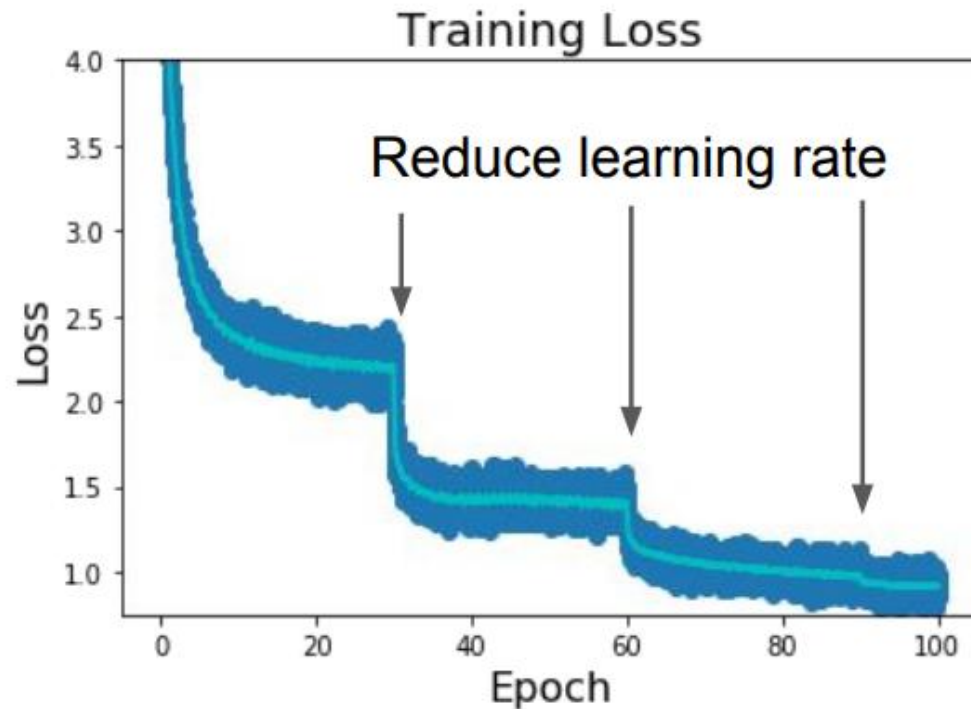
Q: Which one of these learning rates is best to use?

A: Start with large learning rate and decay over time

4. Design choices



Learning rate decay

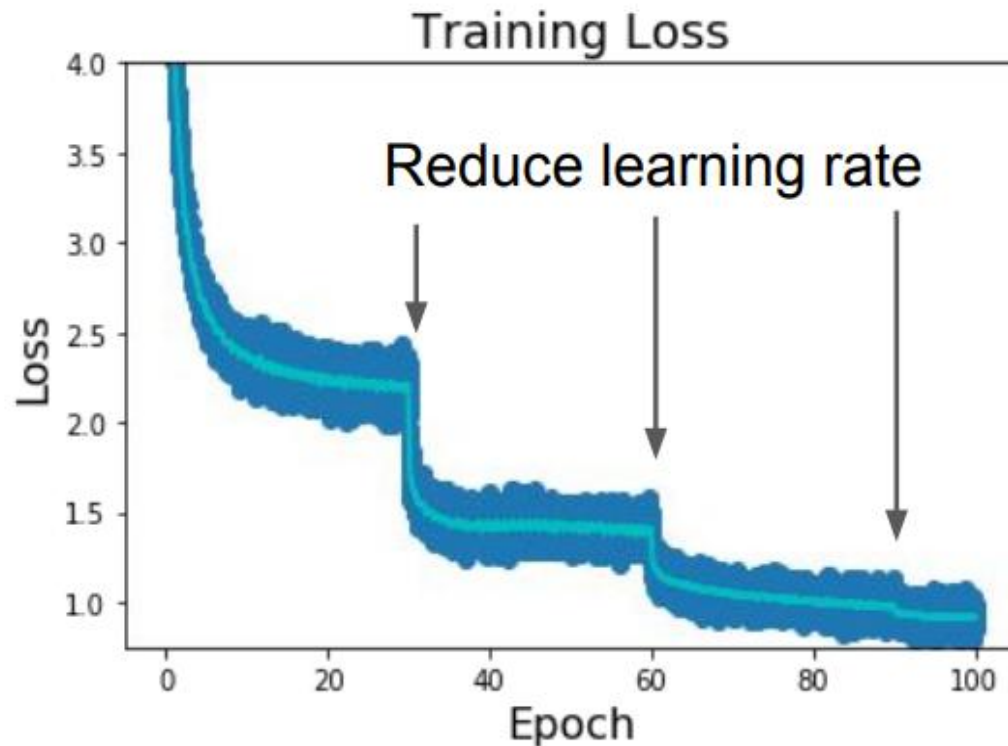


Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

4. Design choices



Learning rate decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Fancy decay schedules like cosine, linear, inverse sqrt.

Empirical rule of thumb: If you increase the batch size by N , also scale the initial learning rate by N

α_0 : Initial learning rate

α_t : Learning rate at epoch t

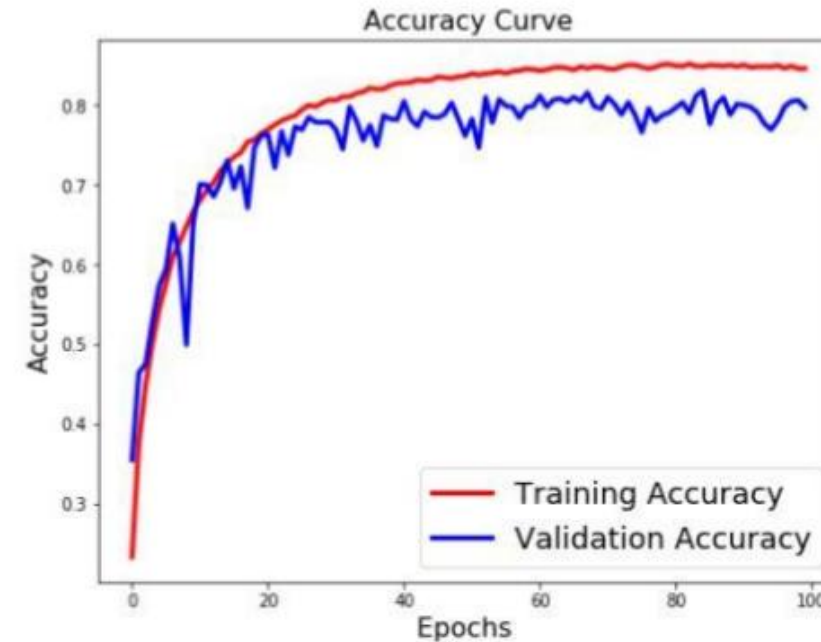
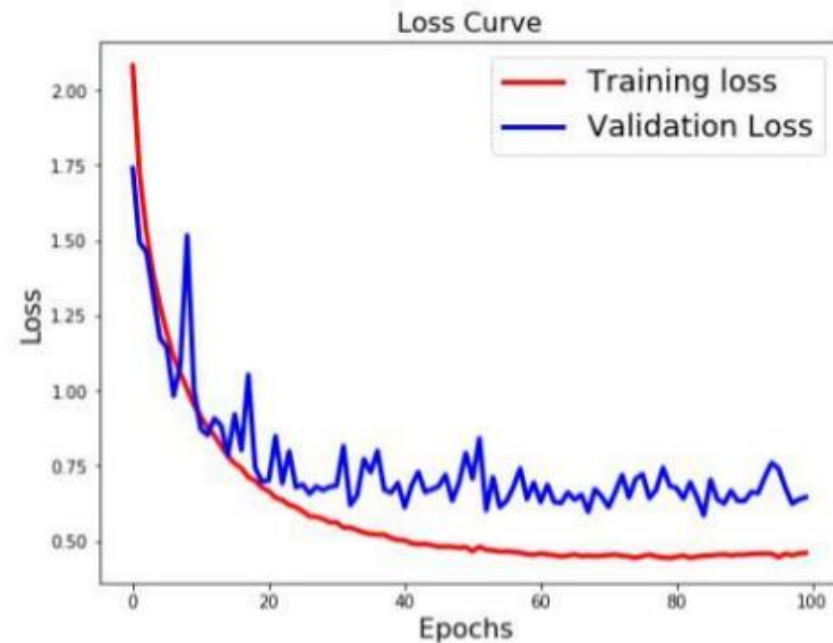
T : Total number of epochs

4. Design choices



Monitor learning curves

Also useful to plot performance on final metric



Periodically evaluate validation loss

Figure credit: <https://www.learnopencv.com/wp-content/uploads/2017/11/cnn-keras-curves-without-aug.jpg>

4. Design choices



Monitor learning curves

Training loss can be noisy. Using a scatter plot or plotting moving average can help better visualize trends.

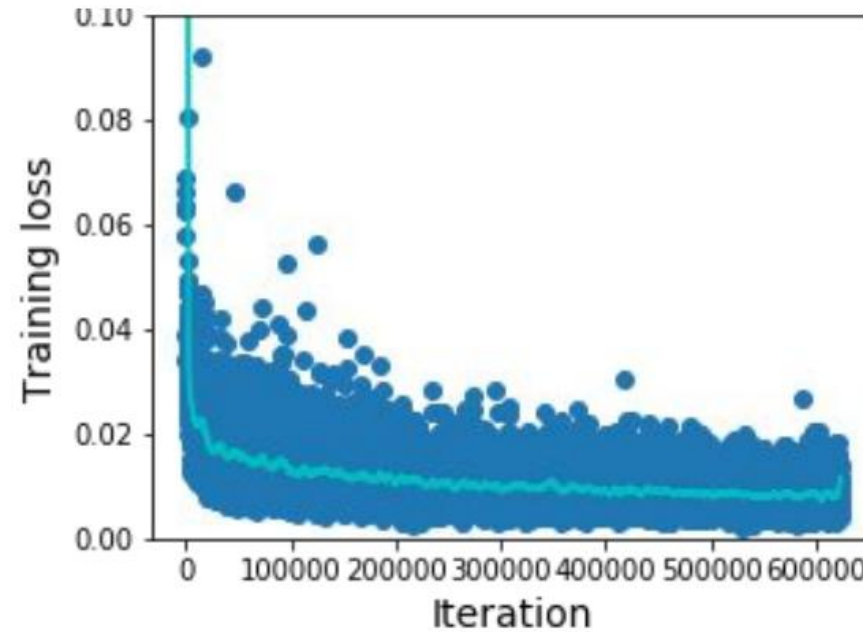
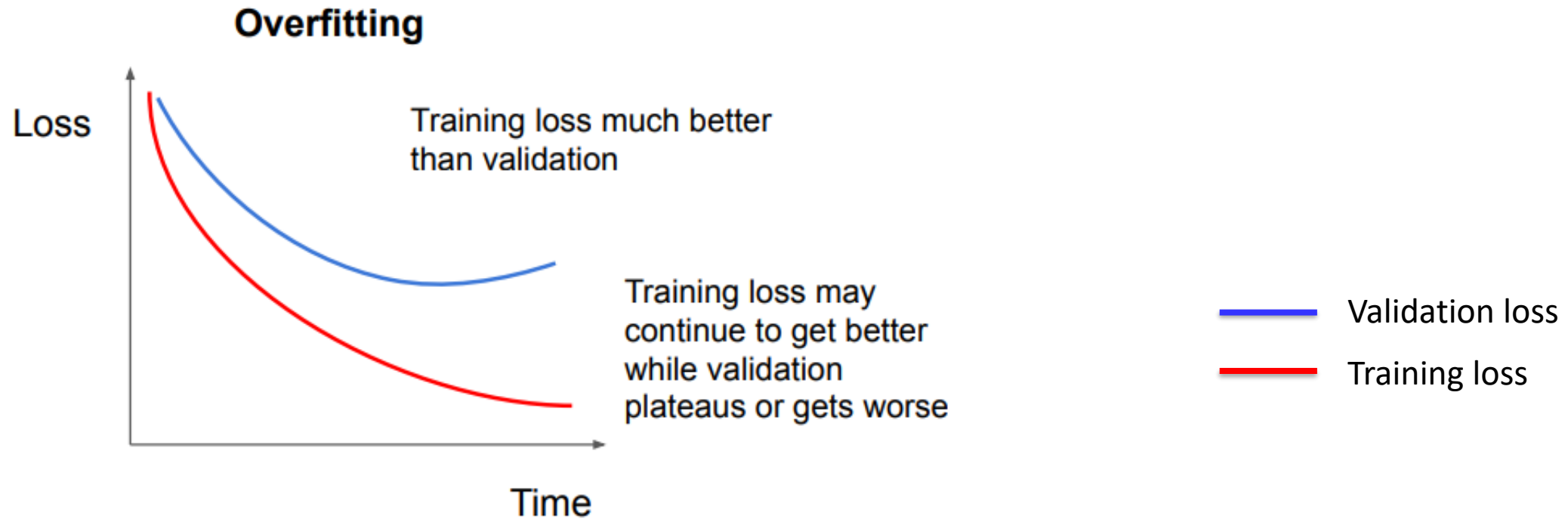


Figure credit: CS231n

4. Design choices



Overfitting vs. underfitting



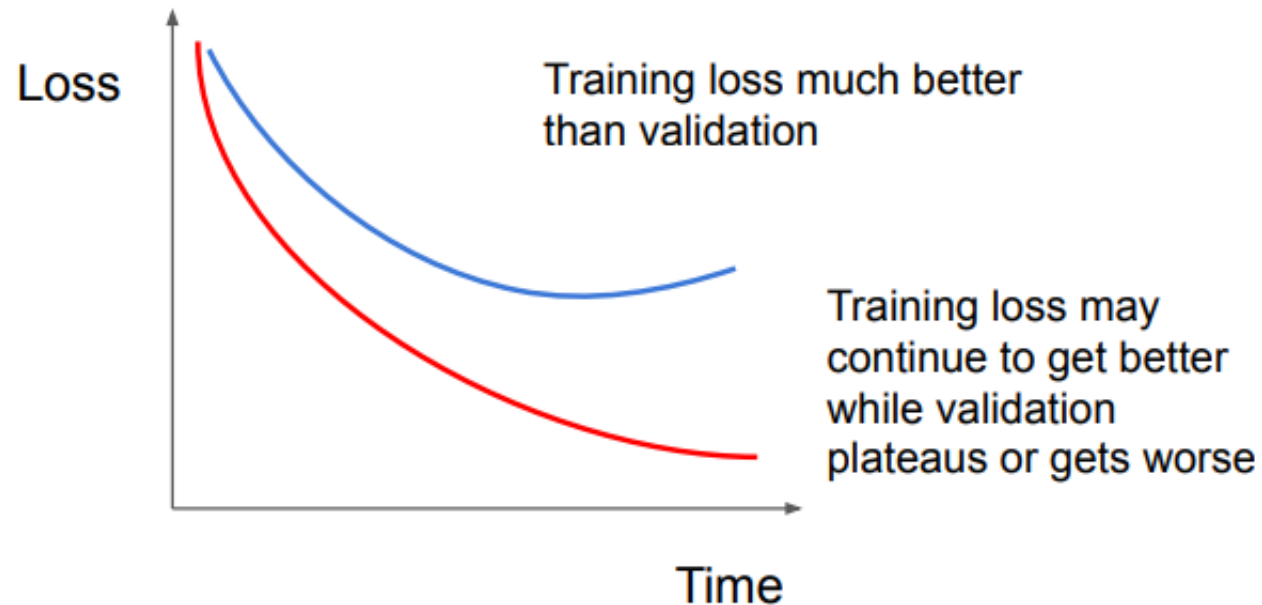
4. Design choices



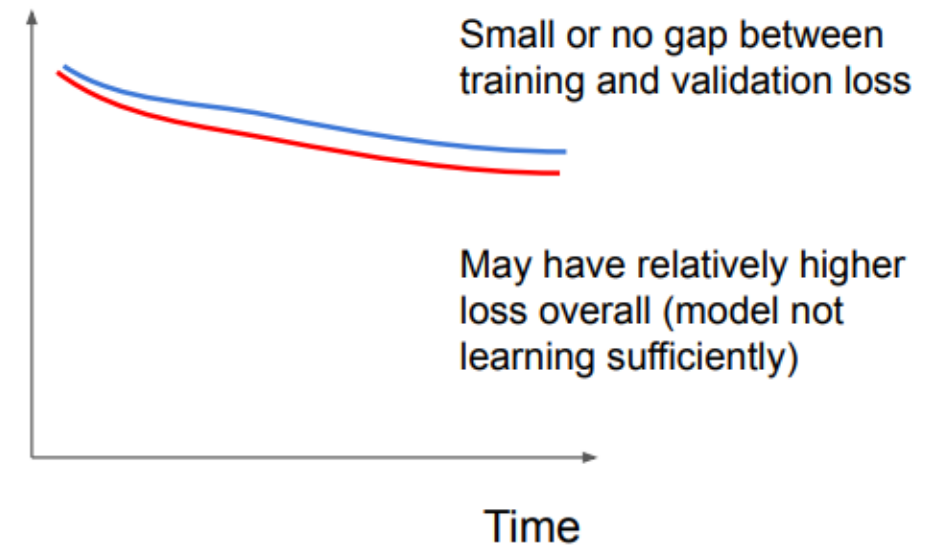
Overfitting vs. underfitting

— Validation loss
— Training loss

Overfitting



Underfitting



4. Design choices



Overfitting vs. underfitting

Question:

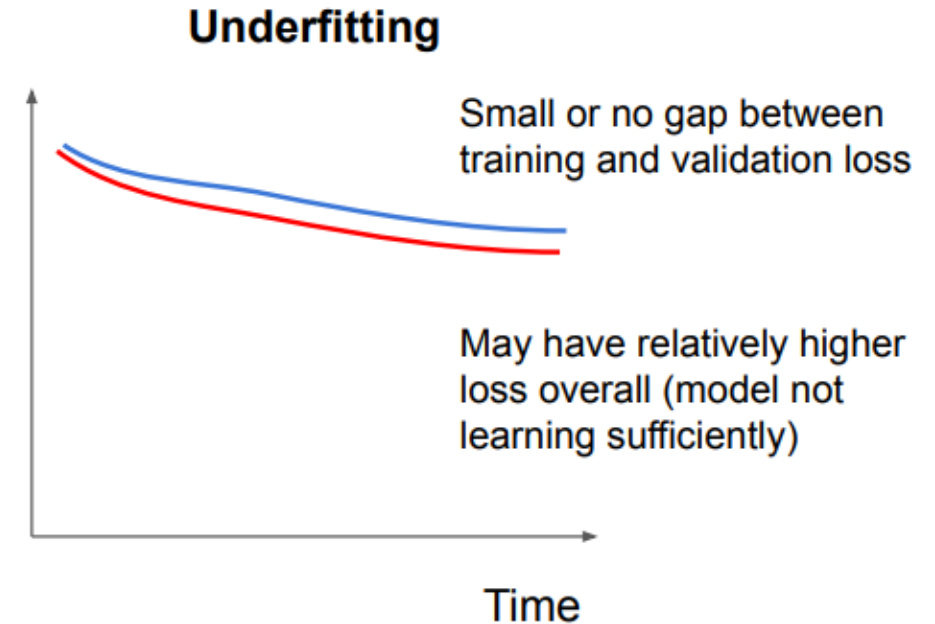
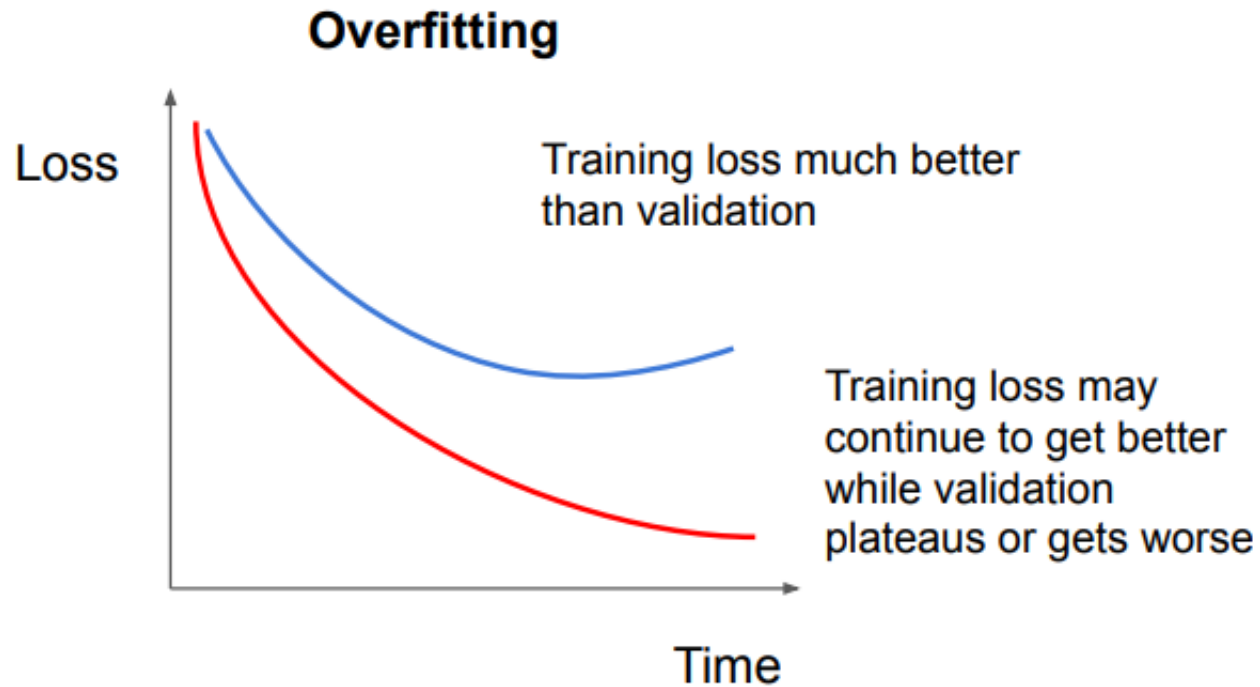
- What are some ways to combat overfitting?
- What are some ways to combat underfitting?

4. Design choices



Overfitting vs. underfitting

— Validation loss
— Training loss

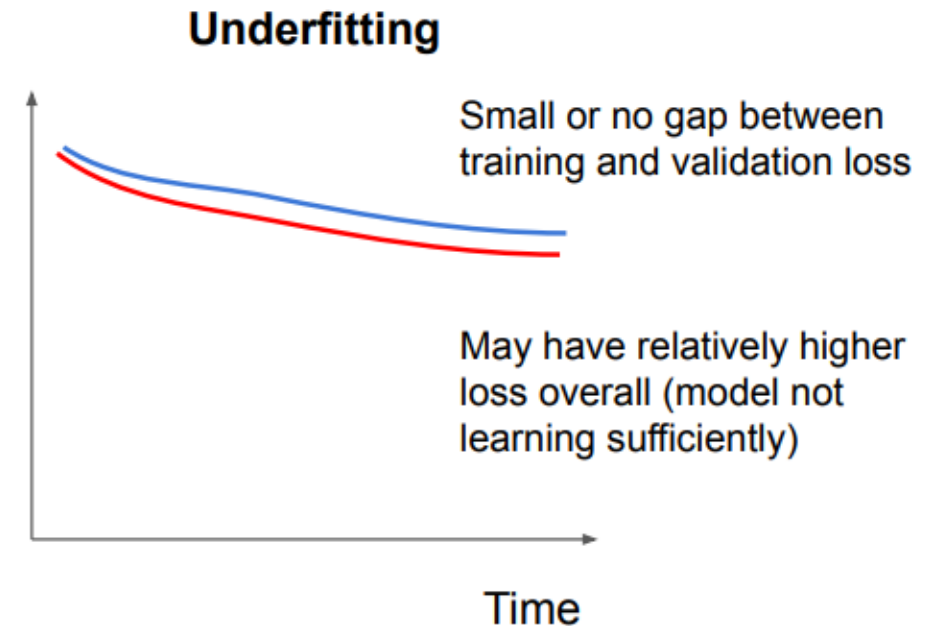
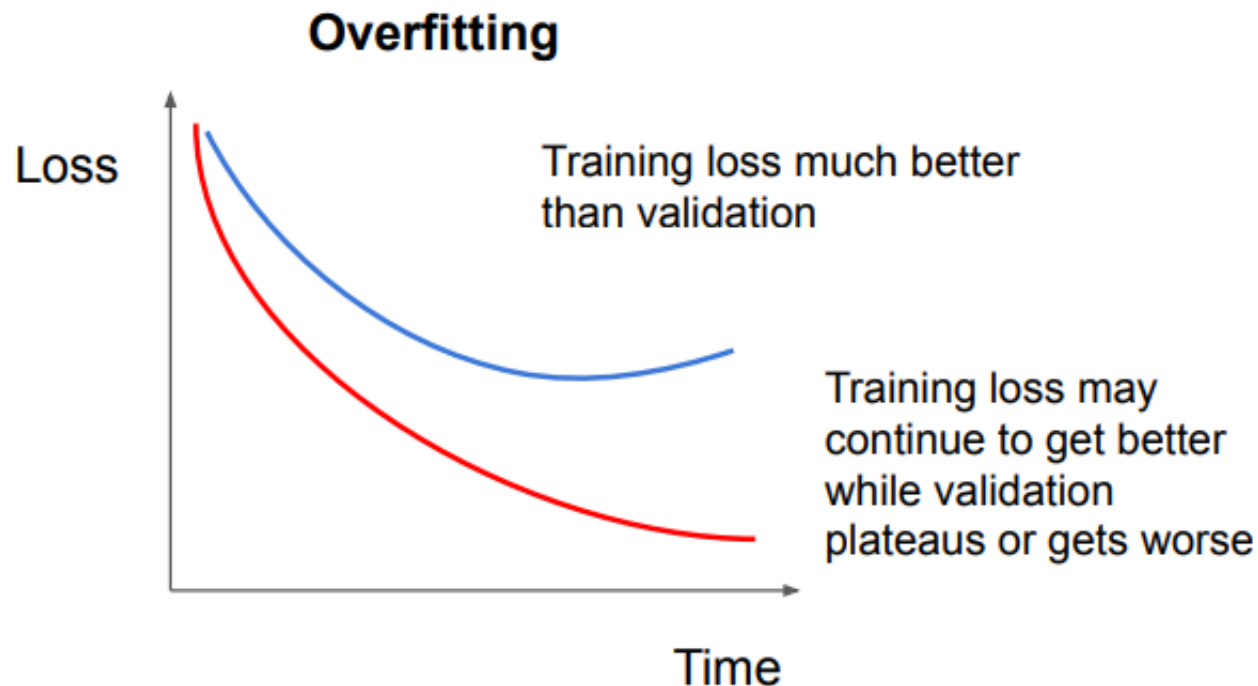


4. Design choices



Overfitting vs. underfitting

— Validation loss
— Training loss



Model is “overfitting” to the training data.

Best strategy: Increase data or regularize model.

Second strategy: Decrease model capacity (make simpler)

Model is not able to sufficiently learn to fit the data well.

Best strategy: Increase complexity (e.g. size) of the model.

Second strategy: make problem simpler (easier task, cleaner data)

4. Design choices



Overfitting vs. underfitting: more intuition

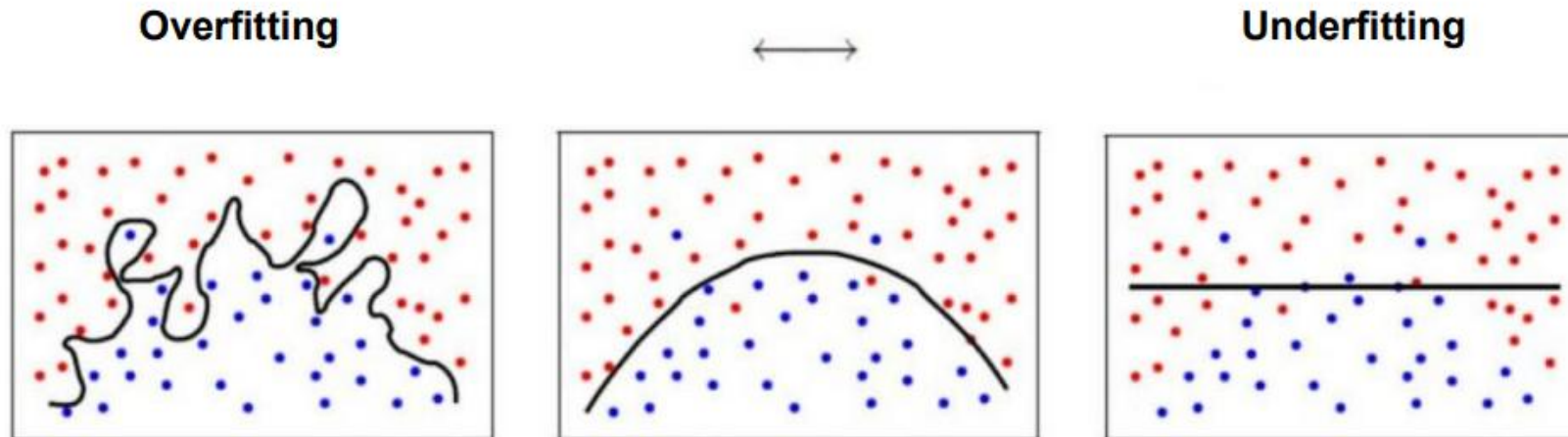
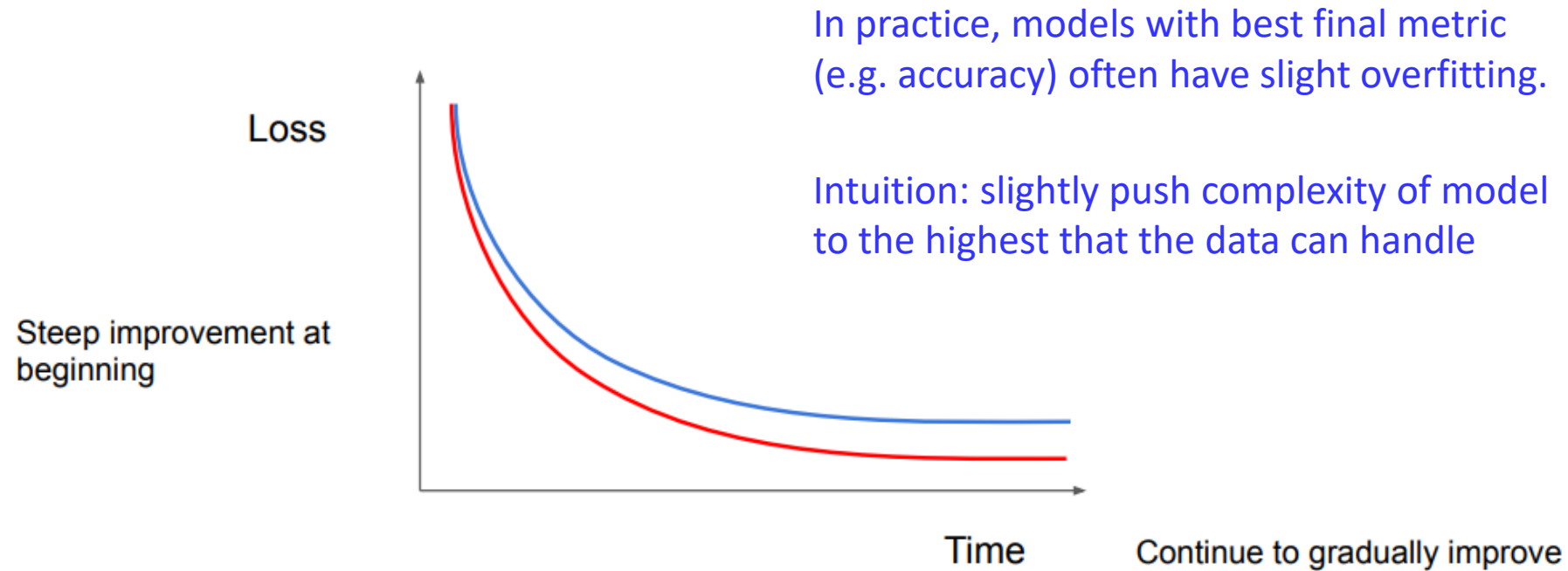


Figure credit: <https://qph.fs.quoracdn.net/main-qimg-412c8556aac7e25b86bba63e9e67ac6-c>

4. Design choices



Healthy learning curves

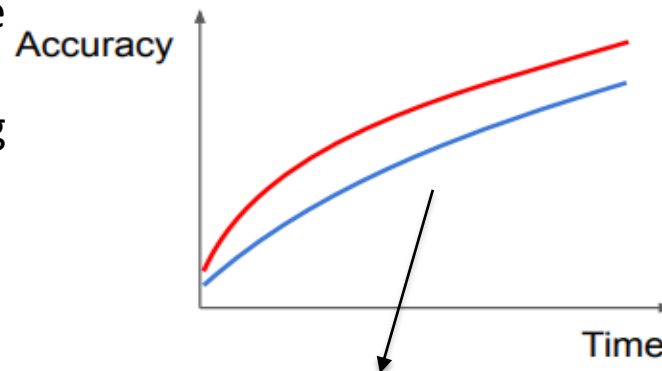
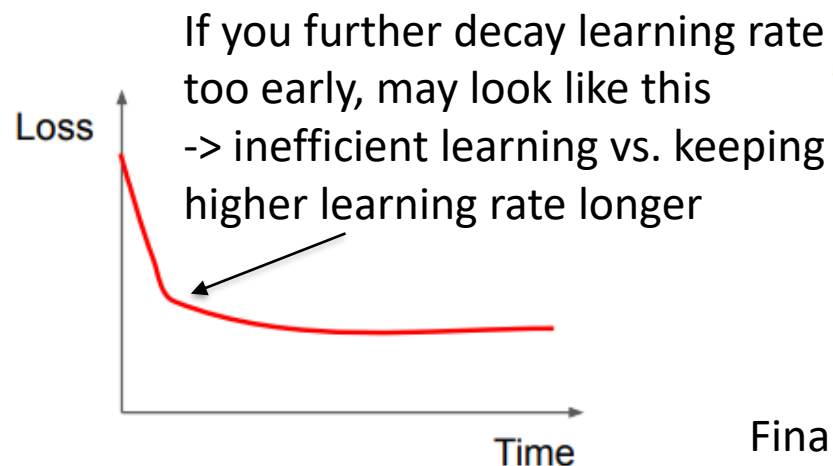
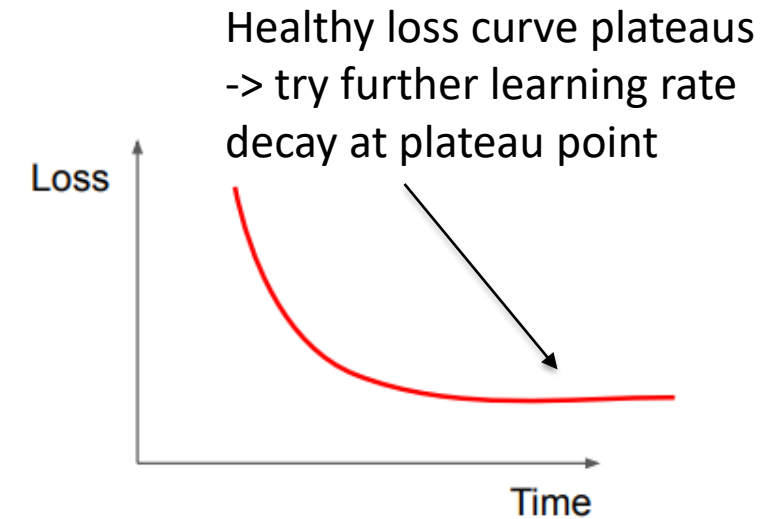
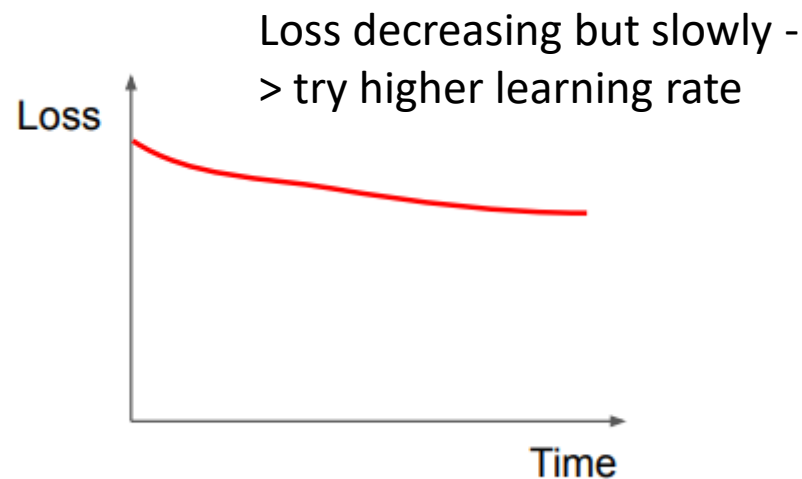
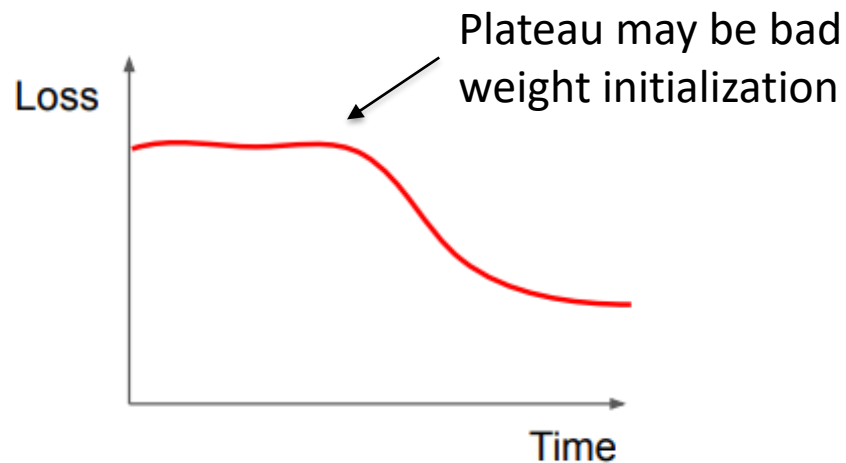


4. Design choices



Model debugging

— Validation loss
— Training loss



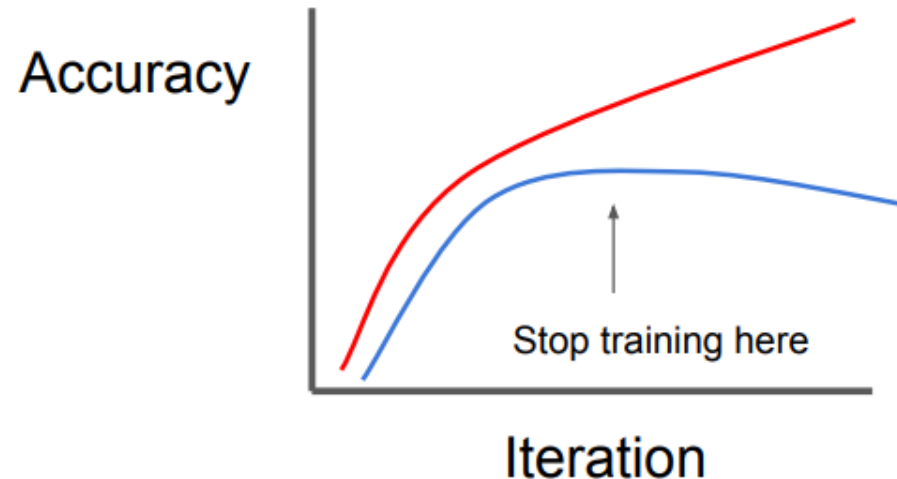
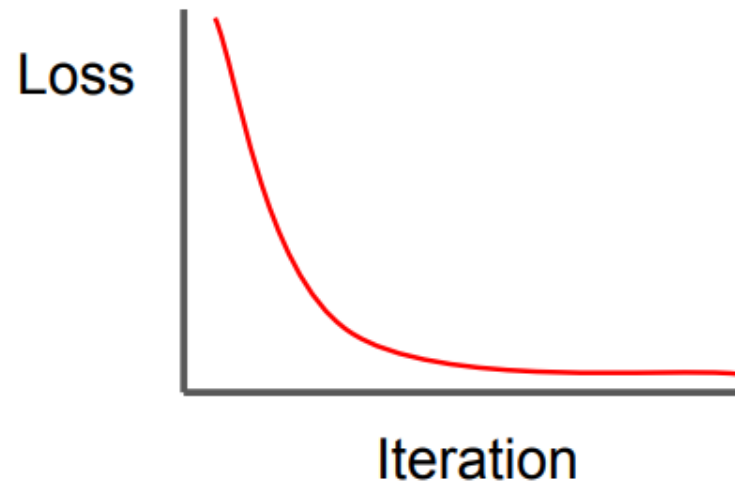
Final metric is still improving -> keep training!

4. Design choices



Early stopping: always do this

— Validation loss
— Training loss

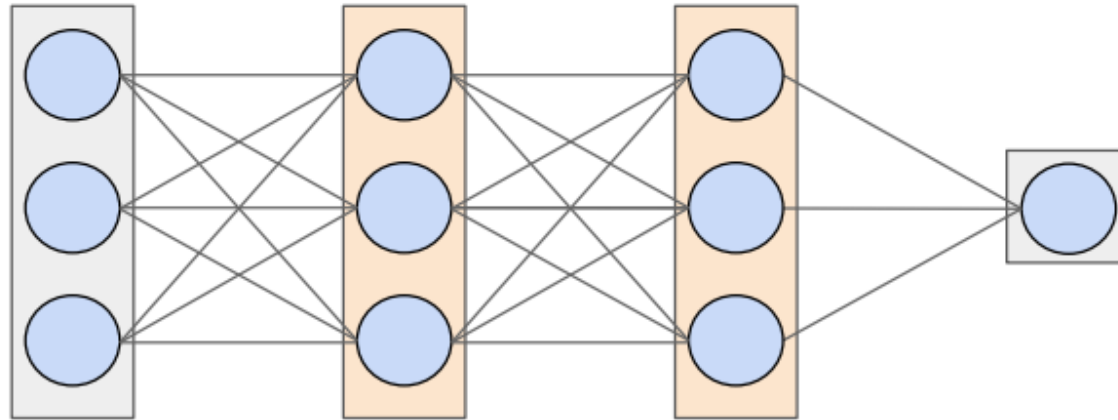


Stop training the model when accuracy on the validation set decreases
Or train for a long time, but always keep track of the model snapshot that worked best on val.

4. Design choices



Design choices: network architectures



If trying to make network bigger (when underfitting) or smaller (when overfitting), network depth and hidden layer size best to adjust first. Don't waste too much time early on fiddling with choices that only minorly change architecture.

Major design choices:

- Architecture type (ResNet, DenseNet, etc. for CNNs)
- Depth (# layers)
- For MLPs, # neurons in each layer (hidden layer size)
- For CNNs, # filters, filter size, filter stride in each layer
- Look at argument options in Tensorflow when defining network layers

4. Design choices



Design choices: regularization (loss term)

Remember optimizing loss functions, which express how well model fit training data, e.g.:

$$L_{\text{regression}} = \frac{1}{M} \sum_i (\hat{y}^i - y^i)^2$$

Regularization adds a term to this, to express preferences on the weights (that prevent it from fitting too well to the training data). Used to combat overfitting:

$$L = \frac{1}{M} \sum_i (\hat{y}^i - y^i)^2 + \lambda R(W)$$

↑ importance of reg. term

↑ Data loss ↑ Regularization loss

Examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$ (weight decay)

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

https://www.tensorflow.org/api_docs/python/tf/keras/regularizers

Remember optimizing loss functions, which express how well model fit training data, e.g.:

Regularization adds a term to this, to express preferences on the weights (that prevent it from fitting too well to the training data). Used to combat overfitting:

L2 most popular: low loss when all weights are relatively small.
More strongly penalizes large weights vs L1.
Expresses preference for simple models (need large coefficients to fit a function to extreme outlier values).

https://www.tensorflow.org/api_docs/python/tf/keras/regularizers

56

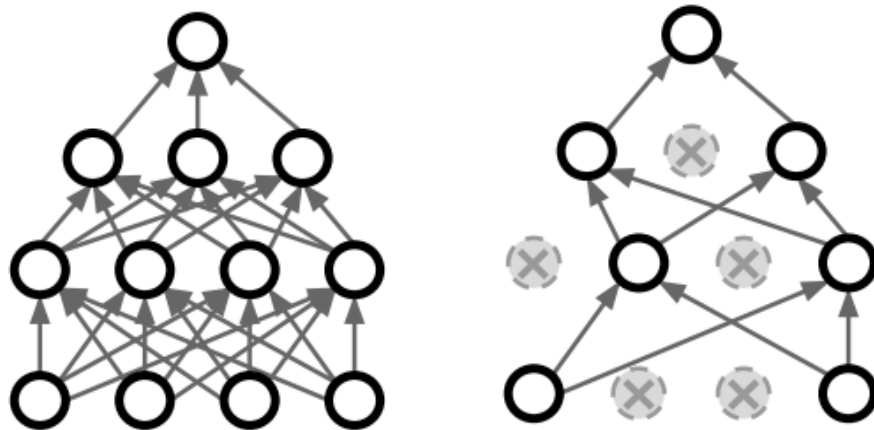
4. Design choices



Design choices: regularization (dropout)

First example of an implicit regularizer.

During training, at each iteration of forward pass randomly set some neurons to zero (i.e., change network architecture such that paths to some neurons are removed).



Probability of “dropping out” each neuron at a forward pass is hyperparameter p . 0.5 and 0.9 are common (high!).

Intuition: dropout is equivalent to training a large ensemble of different models that share parameters.

During testing, all neurons are active. But scale neuron outputs by dropout probability p , such that expected output during training and testing match.

Srivastava et al, “Dropout: A simple way to prevent neural networks from overfitting”, JMLR 2014. Figure credit: CS231n.

4. Design choices



Design choices: regularization (batch normalization)

Another example of an implicit regularizer.

Insert BN layers after FC or conv layers, before activation function.

During training, at each iteration of forward pass normalize neuron activations by mean and variance of minibatch. Also learn scale and shift parameter to get final output.

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

During testing, normalize by a fixed mean and variance computed from the entire training set. Use learned scale and shift parameters.

4. Design choices



Design choices: regularization (batch normalization)

Another example of an implicit regularizer.

Insert BN layers after FC or conv layers, before activation function.

During training, at each iteration of forward pass normalize neuron activations by mean and variance of minibatch. Also learn scale and shift parameter to get final output.

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Intuition: batch normalization allows keeping the weights in a healthy range. Also some randomness at training due to different effect from each minibatch sampling -> regularization!

During testing, normalize by a fixed mean and variance computed from the entire training set. Use learned scale and shift parameters.

4. Design choices



Design choices: data augmentation

Augment effective training data size by simulating more diversity from existing data. Random combinations of:

- Translation and scaling
- Distortion
- Image color adjustment
- Etc.

Think about the domain of your data: what makes sense as realistic augmentation operations?

4. Design choices



Deep learning for healthcare: the rise of medical data

Q: What are other examples of potential data augmentations?

4. Design choices



Design choices: weight initialization

Default initializer for most Keras layers is uniform distribution with a Xavier / Glorot normalization

```
794 @interfaces.legacy_dense_support
795 def __init__(self, units,
796             activation=None,
797             use_bias=True,
798             kernel_initializer='glorot_uniform',
799             bias_initializer='zeros',
800             kernel_regularizer=None,
801             bias_regularizer=None,
802             activity_regularizer=None,
803             kernel_constraint=None,
804             bias_constraint=None,
805             **kwargs):
806     if 'input_shape' not in kwargs and 'input_dim' in kwargs:
807         kwargs['input_shape'] = (kwargs.pop('input_dim'),)
808     super(Dense, self).__init__(**kwargs)
```

Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

4. Design choices



Hyperparameter search

Step 1: Find LR that makes loss go down

Step 2: Define coarse grid of hyperparameter options, train for ~1-5 epochs

Step 3: Refine grid, train longer

Step 4: Look at loss curves

Step 5: GOTO step 3

4. Design choices



Hyperparameter search

Useful debugging / sanity check: restrict to a very small dataset first (e.g. 1 or 2 minibatches). You should be able to severely overfit and drive the loss to 0.



Step 1: Find LR that makes loss go down

Step 2: Define coarse grid of hyperparameter options, train for ~1-5 epochs

Step 3: Refine grid, train longer

Step 4: Look at loss curves

Step 5: GOTO step 3

Common pitfall: making grid too small. Sample a wide range of values to make sure you've explored the space. (e.g. LRs from $1e0$ to $1e-5$.)

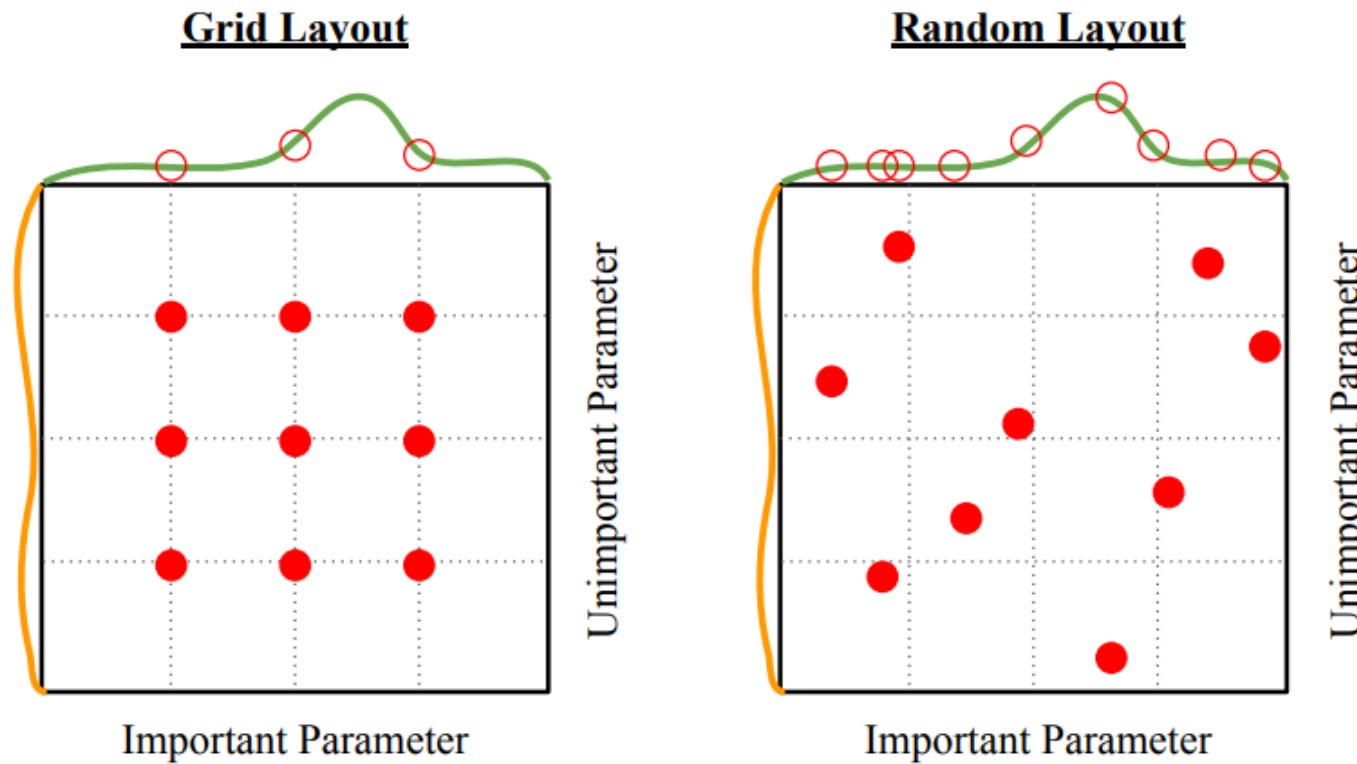
Aside: For LR, should sample e^x for x in Uniform $[-5, 0]$!

4. Design choices



Random search vs. grid search

*Random Search for
Hyper-Parameter Optimization*
Bergstra and Bengio, 2012



Slide credit: CS231n

Illustration of Bergstra et al., 2012 by Shayne
Longpre, copyright CS231n 2017

4. Design choices



Model inference

Maximizing test-time performance: apply data augmentation operations

Main idea: apply model on multiple variants of a data example, and then take average or max of scores

Can use data augmentation operations we saw during training! E.g.:

- Evaluate at different translations and scales
- Common approach for images: evaluate image crops at 4 corners and center, + horizontally flipped versions -> average 10 scores

4. Design choices



Model ensembles

1. Train multiple independent models
2. At test time average their results

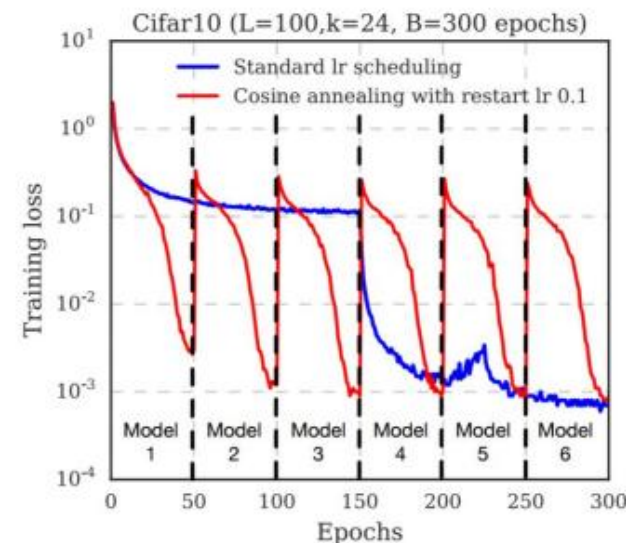
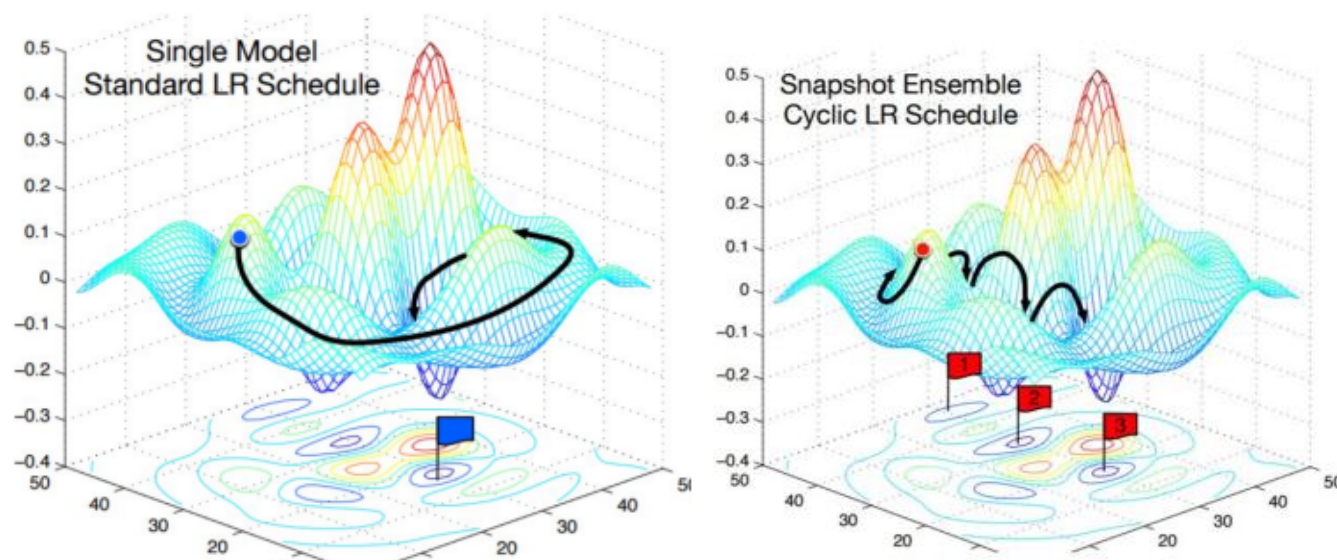
Enjoy 2% extra performance

4. Design choices



Model ensembles: tips and tricks

Instead of training independent models, use multiple snapshots of a single model during training!



Cyclic learning rate schedules can make this work even better!

Loshchilov and Hutter, "SGDR: Stochastic gradient descent with restarts", arXiv 2016
Huang et al, "Snapshot ensembles: train 1, get M for free", ICLR 2017
Figures copyright Yixuan Li and Geoff Pleiss, 2017. Reproduced with permission.

Slide credit: CS231n

4. Design choices



Training more complex neural networks is a straightforward extension

```
keras_model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=3, activation='sigmoid', use_bias=True),
    tf.keras.layers.Dense(units=1, use_bias=True)
])
keras_model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=1e-2),
                    loss='mse')
keras_model.fit(dataset, epochs=1000)
```

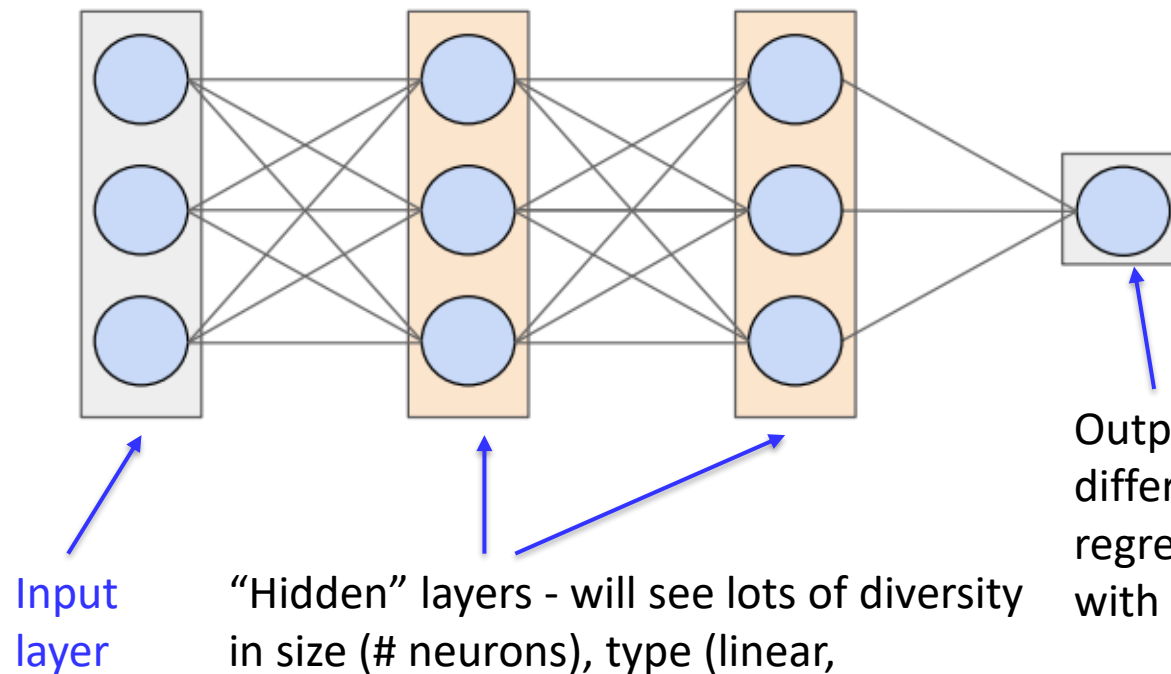
Now a 6-layer network

4. Design choices



“Deep learning”

Can continue to stack more layers to get deeper models!



Vanilla fully-connected neural networks (MLPs) usually pretty shallow -- otherwise too many parameters! ~2-3 layers.

Can have wide range in size of layers (16, 64, 256, 1000, etc.) depending on how much data you have.

Will see different classes of neural networks that leverage structure in data to reduce parameters + increase network depth

4. Design choices



Deep learning

Q: What are other possible tasks besides regression?

4. Design choices



Common activation functions

You can find these in Keras:

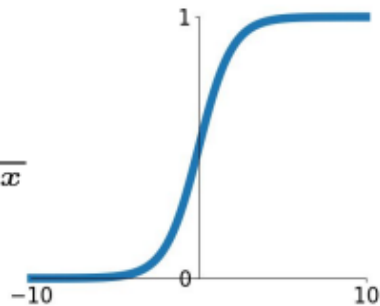
<https://keras.io/layers/advanced-activations/>

You will see these extensively, typically after linear or convolutional layers. They add nonlinearity to allow the model to express complex nonlinear functions.

Typical in modern CNNs and MLPs

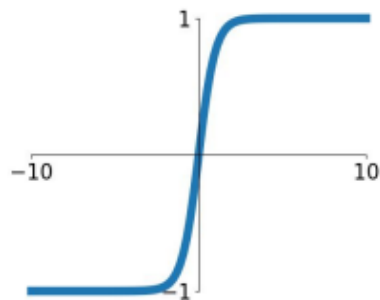
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



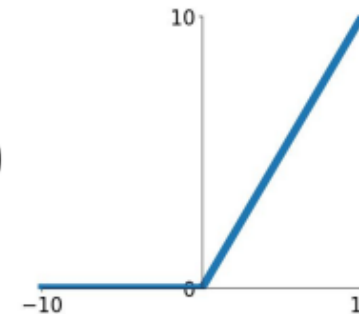
Tanh

$$\tanh(x)$$



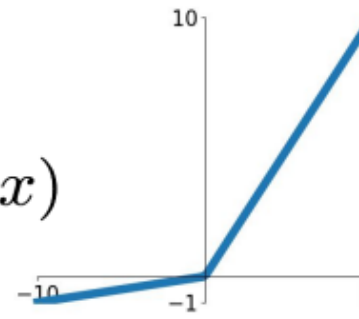
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



and many more...

4. Design choices

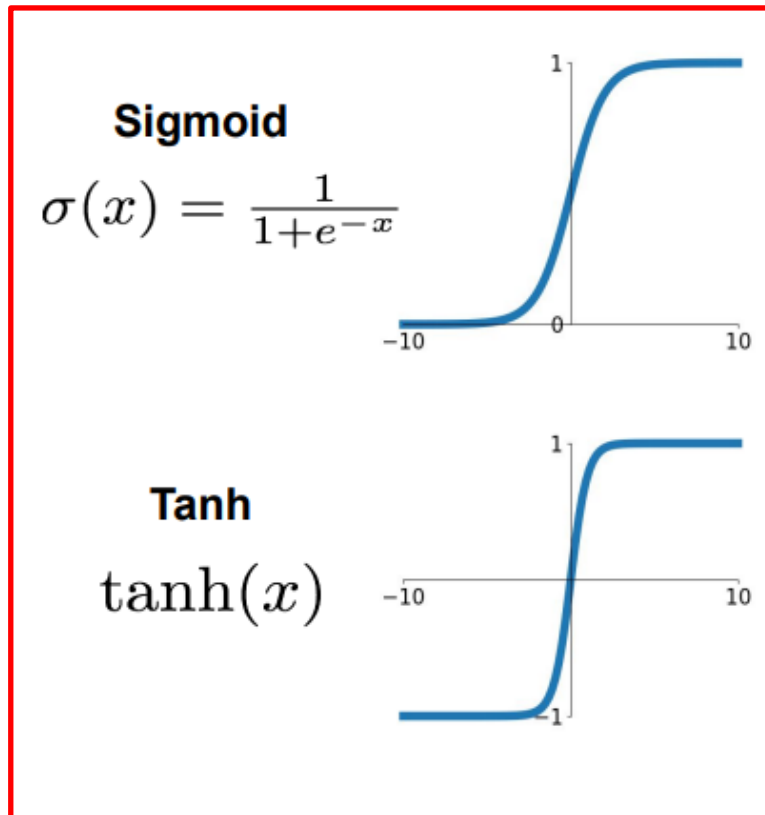


Common activation functions

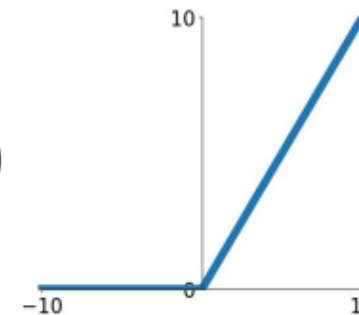
You can find these in Keras:

<https://keras.io/layers/advanced-activations/>

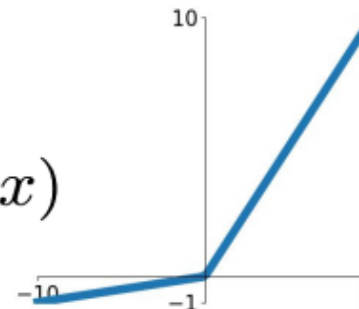
You will see these extensively, typically after linear or convolutional layers. They add nonlinearity to allow the model to express complex nonlinear functions.



ReLU
 $\max(0, x)$



Leaky ReLU
 $\max(0.1x, x)$



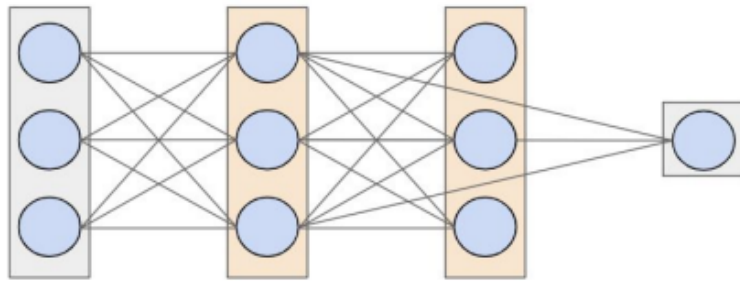
and many more...

Will see in recurrent neural networks. Also used in early MLPs and CNNs.

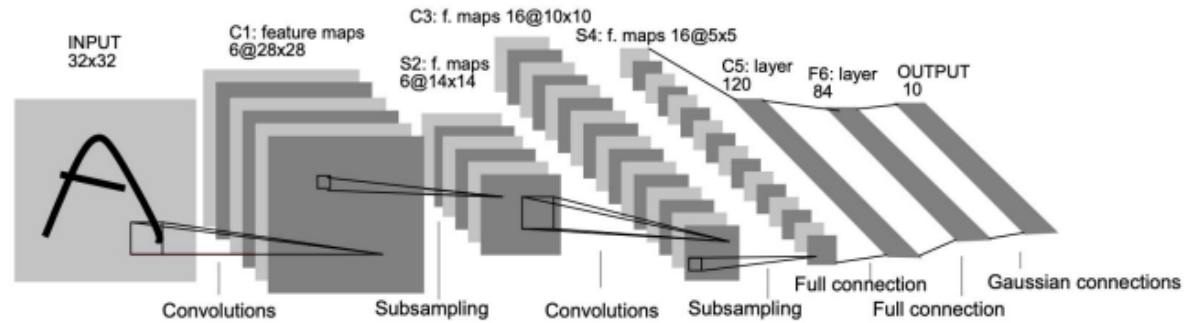
4. Design choices



Will see different classes of neural networks

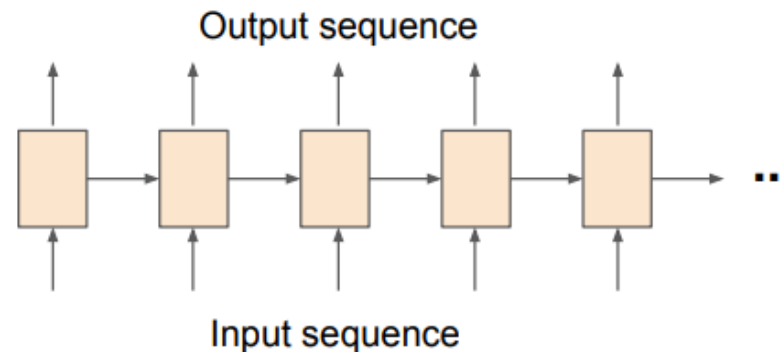


Fully connected neural networks
(linear layers, good for “feature vector” inputs)



Convolutional neural networks
(convolutional layers, good for image inputs)

Recurrent neural networks
(linear layers modeling recurrence relation across sequence, good for sequence inputs)





- Which one can improve the generalization of the model?
 - a. Data augmentation
 - b. Model ensemble
 - c. Dropout strategy
 - d. Add regularization term (loss function)



Summary

- Overview of deep learning fundamentals and training neural networks
- Next class: will go in-depth into
 - Medical image data
 - Classification models
 - Data, model, evaluation considerations